YILDIZ TEKNİK ÜNİVERSİTESİ
KİMYA-METALÜRJİ FAKÜLTESİ
MATEMATİK MÜHENDİSLİĞİ BÖLÜMÜ

MTM4991 – ÇOK DİSİPLİNLİ TASARIM PROJESİ

1. ARA RAPOR

KUBERNETES ORTAMINDA KAYNAK KULLANIMI TAHMİNİ VE TAMSAYILI DOĞRUSAL PROGRAMLAMA İLE MALİYET-OPTİMAL KAPASİTE PLANLAMASI

Danışman : **Doç. Dr. Kadriye ŞİMŞEK ALAN**

Öğrencinin Numarası : **22052063**
Öğrencinin Adı Soyadı : **Berkay DİŞLİ**

Rapor Dönemi : **1.–5. Haftalar (2026–2027 Güz Yarıyılı)**
Teslim Tarihi : **08.11.2026**

İstanbul, 2026

İÇİNDEKİLER

Sayfa

SEMBOL LİSTESİ.....ii
KISALTMA LİSTESİ.....iv
ŞEKİL LİSTESİ.....vi
TABLO LİSTESİ.....vii
ÖZET.....viii
1. GİRİŞ.....1
1.1 Bulut Bilişimde Kapasite Planlama Problemi.....1
1.2 Temel Kavramlar.....2
1.2.1 Kubernetes'te Kaynak Yönetimi.....2
1.2.2 Bulut Sunucu Tipleri ve Fiyatlandırma.....2
1.2.3 Hizmet Düzeyi Anlaşması ve Eksik Kapasite.....3
1.3 Çalışmanın Amacı ve Kapsamı.....3
1.4 Önerilen Yaklaşım ve Yöntem Seçim Gerekçeleri.....4
1.5 Çalışmanın Özgün Değeri.....4
1.6 Raporun Düzeni.....5
2. LİTERATÜR TARAMASI.....6
2.1 İstatistiksel Yöntemlerle Bulut İş Yükü Tahmini.....6
2.2 Derin Öğrenme ile İş Yükü Tahmini.....6
2.3 Kapasite Planlamasında Tamsayılı Programlama.....7
2.4 Proaktif, Reaktif ve Hibrit Otomatik Ölçekleme.....8
2.5 Değerlendirme ve Araştırma Boşluğu.....8
3. METODOLOJİ VE MATEMATİKSEL MODEL.....10
3.1 Genel Sistem Mimarisi.....10
3.2 Veri Kümeleri.....11
3.2.1 Bitbrains GWA-T-12 Veri Kümesi.....11
3.2.2 Alibaba Cluster-Trace-v2018 Veri Kümesi.....12
3.3 Veri Ön İşleme.....13
3.3.1 Zaman Hizalama ve Birim Dönüşümleri.....13
3.3.2 Eksik Veri ve Yaşam Süresi Kuralları.....14
3.3.3 Referans Bellek Varsayımı.....14
3.3.4 Havuz Toplamı ve Saatlik Tepe Talep.....14
3.3.5 Havuz Tasarımı ve Zaman Bölmesi.....15
3.4 Zaman Serisi Tahmin Modelleri.....15
3.4.1 Gün-İleri Tahmin Görevi ve İleri Yürüyen Değerlendirme.....15
3.4.2 Taban Modeller (P0).....16

3.4.3 SARIMA Modeli (P1)....16
3.4.4 LSTM Modeli (P2)....17
3.4.5 Tahmin Başarım Ölçütleri....18
3.5 Kapasite Planlama Modeli....19
3.5.1 Kümeler, Parametreler ve Karar Değişkenleri....19
3.5.2 Amaç Fonksiyonu ve Kısıtlar....20
3.5.3 Kayan Pencere Uygulaması....22
3.5.4 Çözüm Yöntemi ve Doğrulama....22
3.6 Karşılaştırma Stratejileri....22
3.7 Değerlendirme Ölçütleri....25
3.8 Duyarlılık Analizi ve Sentetik Senaryo Tasarımı....26
4. İLK BEŞ HAFTADA TAMAMLANAN ÇALIŞMALAR VE ÖN SONUÇLAR....28
4.1 Tamamlanan Çalışmalar....28
4.2 Keşifsel Veri Analizi....29
4.3 ILP Modelinin İlk Uygulaması....31
4.4 Karşılaşılan Sorunlar ve Çözüm Yaklaşımları....33
5. SONRAKİ AŞAMALAR İÇİN ÇALIŞMA PLANI....34
KAYNAKLAR....35
EK A PROJE YAZILIMININ YAPISI VE ILP MODELİ KODU....38

SEMBOL LİSTESİ

B	Geri kaydırma işleci ($B\, y_t = y_{t-1}$)
b_t	t dilimindeki tampon sunucu sayısı (R1, R2)
C_k	k tipi sunucunun işlemci kapasitesi (vCPU)
c_k	k tipi sunucunun saatlik maliyeti (USD/saat)
D^5_τ	τ 5 dakikalık hücresindeki havuz toplam talebi
$\bar{D}_h$	h saatindeki ortalama havuz talebi
D_h	h saatindeki saatlik tepe havuz talebi
D^{cpu}_t, D^{mem}_t	t dilimindeki gerçekleşen saatlik tepe işlemci (çekirdek) ve bellek (GB) talebi
$\hat{D}^{cpu}_t$, $\hat{D}^{mem}_t$	t dilimi için tahmin edilen saatlik tepe işlemci ve bellek talebi
d_k	k tipi bir sunucuyu kapatma maliyeti
ESO	Eksik kapasiteli saat oranı
ETO	Eksik tahmin oranı
F_s	STL ayrışımına dayalı mevsimsellik gücü
g_t	t diliminde gerekli tampon sunucu sayısı
H	Planlama ufku (saatlik dilim sayısı)
h	Saat indisi; tahmin adımı (Eşitlik 3.8–3.9)
K	Sunucu tipleri kümesi
k	Sunucu tipi indisi
kap^r_t	t dilimindeki r kaynağı kapasitesi ($r \in \{cpu, mem\}$)
L_k	k tipi sunucunun minimum çalışma süresi (dilim)
M_k	k tipi sunucunun bellek kapasitesi (GiB)
M_{ref}	Alibaba makineleri için referans bellek miktarı (GB)
N	Test dönemindeki tahmin dilimi sayısı
n	Pod (kopya) sayısı
o	Tahmin kökeni (günün ilk saati)
$(p, d, q)\times(P, D, Q)_s$	SARIMA modelinin mevsimsel olmayan ve mevsimsel dereceleri (s: mevsim uzunluğu)
q^r	Ölçek tipinin (m8i.xlarge) r kaynağındaki kapasitesi
r	Referans bellek katsayısı (GB/çekirdek); Bölüm 3.6–3.7'de kaynak indisi (cpu, mem)
s_k	k tipi bir sunucuyu açma (başlatma) maliyeti
t	Zaman dilimi (saat) indisi

T	Değerlendirme dönemindeki dilim sayısı
u_t	t dilimindeki kapasite kullanım oranı
V	Havuzdaki makinelerin kümesi
w	Açma maliyeti katsayısı (ısınma süresinin saat eşdeğeri)
$\hat{y}_t$	t dilimi için tahmin edilen değer
$x_{k,t}$	t diliminde çalışan k tipi sunucu sayısı
x^0_k	Planlama ufkunun başında çalışan k tipi sunucu sayısı
$y_{k,t}$	t diliminin başında açılan k tipi sunucu sayısı
$z_{k,t}$	t diliminin başında kapatılan k tipi sunucu sayısı
α	Güvenlik payı
ε_t	Beyaz gürültü (SARIMA hata terimi)
ρ^{cpu}, ρ^{mem}	Kullanılabilir işlemci ve bellek kapasitesi oranı
σ	Lojistik (sigmoid) fonksiyon
$\tau_{üst}$, τ_{hedef}, τ_{alt}	Eşik kuralının üst, hedef ve alt kullanım eşikleri
φ_p, θ_q, Φ_P, Θ_Q	SARIMA modelinin otoregresif ve hareketli ortalama çok terimlileri

KISALTMA LİSTESİ

ADF	Augmented Dickey–Fuller (Genişletilmiş Dickey–Fuller Testi)
AIC	Akaike Information Criterion (Akaike Bilgi Ölçütü)
ARIMA	Autoregressive Integrated Moving Average (Otoregresif Bütünleşik Hareketli Ortalama)
ARMA	Autoregressive Moving Average (Otoregresif Hareketli Ortalama)
AWS	Amazon Web Services
CP-SAT	Constraint Programming – Satisfiability (Kısıt Programlama – Sağlanabilirlik Çözücüsü)
EC2	Elastic Compute Cloud
GiB	Gibibayt
GLOP	Google Linear Optimization Package (Google Doğrusal Optimizasyon Paketi)
GPU	Graphics Processing Unit (Grafik İşlem Birimi)
HPA	Horizontal Pod Autoscaler (Yatay Pod Otomatik Ölçekleyicisi)
ILP	Integer Linear Programming (Tamsayılı Doğrusal Programlama)
KPSS	Kwiatkowski–Phillips–Schmidt–Shin Durağanlık Testi
LP	Linear Programming (Doğrusal Programlama)
LSTM	Long Short-Term Memory (Uzun Kısa Süreli Bellek)
MAE	Mean Absolute Error (Ortalama Mutlak Hata)
MAPE	Mean Absolute Percentage Error (Ortalama Mutlak Yüzde Hata)
MASE	Mean Absolute Scaled Error (Ortalama Mutlak Ölçeklenmiş Hata)
MSE	Mean Squared Error (Ortalama Kare Hata)
RMSE	Root Mean Square Error (Hata Kareleri Ortalamasının Karekökü)
RNN	Recurrent Neural Network (Tekrarlayan Sinir Ağı)
SaaS	Software as a Service (Hizmet Olarak Yazılım)
SARIMA	Seasonal Autoregressive Integrated Moving Average (Mevsimsel Otoregresif Bütünleşik Hareketli Ortalama)
SCIP	Solving Constraint Integer Programs
SLA	Service Level Agreement (Hizmet Düzeyi Anlaşması)
STL	Seasonal-Trend Decomposition using Loess (Loess ile Mevsim-Eğilim Ayrışımı)
vCPU	Virtual CPU (Sanal İşlemci)

VM	Virtual Machine (Sanal Makine)
VPA	Vertical Pod Autoscaler (Dikey Pod Otomatik Ölçekleyicisi)

ŞEKİL LİSTESİ

Şekil 3.1 Önerilen kapasite planlama sisteminin genel mimarisi.....11
Şekil 3.2 R2 hibrit stratejisinde tamponun güncellenmesi.....24
Şekil 4.1 Bitbrains fastStorage havuzunun saatlik tepe ve saatlik ortalama işlemci ve bellek talebi.....29
Şekil 4.2 Bitbrains havuzlarında bellek tepe talebinin işlemci tepe talebine oranının dağılımı (eğitim dönemi).....30
Şekil 4.3 Bitbrains fastStorage havuzunda bir günlük ILP planı: sunucu sayıları ile kapasite ve güvenlik paylı SARIMA tahmini.....32

TABLO LİSTESİ

Tablo 2.1 İncelenen çalışmaların bu çalışmanın bileşenleri açısından karşılaştırılması....9
Tablo 3.1 Veri kümelerinin ve havuzların özellikleri....12
Tablo 3.2 LSTM modelinin yapısı ve eğitim ayarları....18
Tablo 3.3 Sunucu tipleri ve özellikleri (AWS EC2 on-demand, Linux, eu-central-1)....20
Tablo 3.4 ILP modelinin parametreleri....21
Tablo 3.5 Karşılaştırma stratejileri ve eşik parametreleri....24
Tablo 3.6 Sentetik senaryolar....27
Tablo 4.1 Toplam saatlik tepe talebin toplam saatlik ortalama talebe oranı....30
Tablo 4.2 Talebin bellek/işlemci oranı (GB/çekirdek) ve bellek kısıtının bağlayıcı olduğu saatlerin oranı (eğitim dönemi)....31
Tablo 5.1 6.–15. haftalar için çalışma planı....34

ÖZET

Bulut ortamlarında çalışan uygulamalar için ayrılan kapasitenin talebin üzerinde kalması gereksiz maliyete, altında kalması ise hizmet düzeyi ihlallerine yol açar. Bu çalışmada, bir Kubernetes düğüm havuzunun işlemci ve bellek talebini bir gün öncesinden tahmin eden ve bu tahmine göre maliyeti en küçük sunucu planını tamsayılı doğrusal programlama (ILP) ile belirleyen bir kapasite planlama sistemi tasarlanmaktadır. Bu rapor, çalışmanın ilk beş haftasını kapsayan birinci ara rapordur.

İlk beş haftada çalışmanın kapsamı belirlenmiş; bulut iş yükü tahmini, kapasite planlamasında tamsayılı programlama ve otomatik ölçekleme alanlarında literatür taraması yapılmıştır. Bitbrains GWA-T-12 ve Alibaba cluster-trace-v2018 veri kümeleri 5 dakikalık ortak bir ızgaraya hizalanmış ve havuz toplamının saatlik tepesi planlama talebi olarak tanımlanmıştır. Tahmin için mevsimsel naif ve son değer taban modelleri, SARIMA ve LSTM modelleri kurulmuştur. Üç AWS EC2 sunucu tipi üzerinden, açma maliyeti ve minimum çalışma süresi kısıtları içeren çok dönemli bir ILP modeli formüle edilmiş ve Google OR-Tools aracılığıyla SCIP çözücüsüyle çözülecek biçimde kodlanmıştır.

Keşifsel analizde saat içi tepe talebin saatlik ortalama talebi 1,02–1,37 kat aştığı ve bu nedenle planlamanın tepe talebe göre yapılması gerektiği görülmüştür. Bitbrains havuzlarında işlemcinin, Alibaba havuzlarında ise belleğin bağlayıcı kaynak olduğu belirlenmiştir. ILP modelinin ilk uygulamasında bir günlük plan 0,1 saniyenin altında en iyi çözüme ulaşmıştır. Sonraki aşamalarda tahmin ve planlama sonuçlarının test döneminde değerlendirilmesi, tepkisel ve hibrit stratejilerle karşılaştırma, duyarlılık analizi ve sentetik senaryo deneyleri yapılacaktır.

Anahtar Kelimeler: Kapasite planlama, tamsayılı doğrusal programlama, zaman serisi tahmini, Kubernetes, otomatik ölçekleme, bulut bilişim.

1. GİRİŞ

Bu bölümde, bulut ortamlarında kapasite planlama probleminin arka planı ve raporun izlenebilmesi için gerekli temel kavramlar verilmiştir. Ardından çalışmanın amacı ve kapsamı, izlenen yaklaşım ve yöntem seçimlerinin gerekçeleri ile çalışmanın özgün değeri açıklanmış, son olarak raporun düzeni tanıtılmıştır.

1.1 Bulut Bilişimde Kapasite Planlama Problemi

Bulut bilişim, işlem gücü ve bellek gibi kaynakların ihtiyaç duyulduğu anda kiralanmasına ve yalnızca kullanıldığı süre kadar ücretlendirilmesine olanak tanır [1]. Bu esneklik, kaynakların talebe göre artırılıp azaltılabilmesi anlamına gelir; ancak hangi anda ne kadar kaynak ayrılacağı kararı kullanıcıya kalır. Günümüzde bulut uygulamalarının önemli bir bölümü konteynerler içinde çalıştırılmakta, bu konteynerlerin sunuculara yerleştirilmesi, ölçeklenmesi ve yönetimi ise Kubernetes gibi orkestrasyon sistemleriyle yapılmaktadır [2].

Kapasite planlamasında iki tür hata yapılabilir. Talebin üzerinde kapasite ayrılması (aşırı ayırma, over-provisioning) boşta kalan sunucular için gereksiz ödeme yapılmasına yol açar. Büyük ölçekli kümelerde ayrılan kaynakların önemli bir kısmının fiilen kullanılmadığı gözlemlenmiştir [3]. Talebin altında kapasite ayrılması (eksik ayırma, under-provisioning) ise isteklerin gecikmesine, hizmet düzeyi hedeflerinin ihlal edilmesine ve servis kesintisi riskine neden olur. Bu nedenle kapasite planlaması, maliyet ile hizmet sürekliliği arasında bir denge problemidir.

Planlamanın hangi talep ölçüsüne göre yapılacağı da bu dengeyi doğrudan etkiler. Kaynak kullanımı dakikalar içinde dalgalandığından, saatlik ortalama kullanıma göre ayrılan bir kapasite saat içindeki kısa süreli zirveleri karşılayamaz. Bu çalışmada kullanılan veri kümelerinde, saat içindeki en yüksek 5 dakikalık havuz talebinin saatlik ortalama talebe oranı, toplamlar üzerinden 1,02 ile 1,37 arasında bulunmuştur (Bölüm 4.2). Örneğin Alibaba kümesinde işlemci için bu oran 1,171'dir; başka bir deyişle saatlik ortalamaya göre yapılan bir planlamada, saat içi tepe anlarında talebin ortalama olarak yaklaşık %15'i ($1 - 1/1{,}171$) karşılanamayacaktır. Bu gözlem, çalışmada kapasite planlamasının saatlik tepe talep üzerinden yapılmasının gerekçesini oluşturmuştur.

Kubernetes'in ve bulut sağlayıcılarının sunduğu otomatik ölçekleme mekanizmaları büyük ölçüde tepkiseldir: kaynak kullanımı belirli bir eşiği aştıktan sonra kapasite artırılır [4, 5]. Tepkisel yaklaşım basit ve sağlamdır; ancak karar talep arttıktan sonra verildiği için bir gecikme içerir ve yeni bir sunucunun açılıp hizmete girmesi de zaman alır. Talebin önceden tahmin edilmesine dayanan proaktif yaklaşımlar bu gecikmeyi azaltmayı amaçlar [6, 7]. Proaktif bir planın maliyet açısından en iyi olabilmesi için ise tahmin edilen talebi karşılayacak sunucuların sayısı ve tipleri, sunucu sayılarının tamsayı olduğu dikkate alınarak belirlenmelidir. Bu çalışmada bu iki ihtiyaç, zaman serisi tahmini ile tamsayılı doğrusal programlamanın (ILP) birleştirilmesiyle ele alınmıştır.

1.2 Temel Kavramlar

Bu alt bölümde, raporun izlenebilmesi için gerekli olan Kubernetes kaynak yönetimi, bulut sunucu tipleri ve fiyatlandırma ile hizmet düzeyi kavramları kısaca tanıtılmıştır.

1.2.1 Kubernetes'te Kaynak Yönetimi

Kubernetes'te çalıştırılan en küçük birim, bir veya daha fazla konteynerden oluşan pod'dur. Her pod için işlemci ve bellek bakımından bir istek (request) ve bir sınır (limit) tanımlanabilir. İstek, pod'un yerleştirileceği sunucuda kendisi için ayrılacak kaynak miktarını; sınır ise pod'un kullanabileceği en yüksek kaynak miktarını belirtir [8]. Pod'lar, düğüm (node) adı verilen sanal veya fiziksel sunucular üzerinde çalışır. Aynı tipteki düğümlerin oluşturduğu gruba düğüm havuzu (node pool) denir ve bir düğüm havuzunun maliyeti, içindeki düğümlerin sayısı ve tipiyle belirlenir.

Kubernetes'te ölçekleme üç düzeyde yapılır. Yatay Pod Otomatik Ölçekleyicisi (HPA), izlenen bir ölçütün, örneğin ortalama işlemci kullanımının, hedef değere oranına göre pod sayısını değiştirir [4]. Mevcut pod sayısı n, gözlenen kullanım oranı u ve hedef kullanım oranı τ_{hedef} ile gösterildiğinde, yeni pod sayısı Eşitlik 1.1'deki gibi hesaplanır.

$$n^{yeni} = \lceil n \cdot \frac{u}{\tau_{hedef}} \rceil \tag{1.1}$$

Burada u son ölçüm penceresinde gözlenen değerdir; dolayısıyla HPA talepteki bir artışa ancak artış gözlendikten sonra tepki verir. Dikey Pod Otomatik Ölçekleyicisi (VPA) pod sayısını değil, pod'ların istek değerlerini geçmiş kullanıma göre ayarlar. Google'ın Autopilot sistemi de benzer biçimde, geçmiş kullanım pencerelerinin yüksek yüzdeliklerine ve tepe değerlerine dayanarak istek değerleri önerir [9]. Düğüm düzeyinde ise küme otomatik ölçekleyicisi (Cluster Autoscaler), mevcut düğümlere sığmayan pod'lar olduğunda düğüm havuzuna yeni düğüm ekler ve uzun süre düşük kullanımda kalan düğümleri kaldırır [10]. Bu çalışmada ele alınan karar, bu son düzeye, yani düğüm havuzunda hangi tipten kaç sunucunun çalıştırılacağına karşılık gelir.

1.2.2 Bulut Sunucu Tipleri ve Fiyatlandırma

Bulut sağlayıcıları sunucuları, sanal işlemci (vCPU) sayısı ve bellek miktarı sabit olan sunucu tipleri (instance types) olarak sunar. Amazon Web Services (AWS) EC2 hizmetinde aynı nesildeki tipler, işlemci başına düşen bellek miktarına göre ailelere ayrılır: hesaplama optimize (c), genel amaçlı (m) ve bellek optimize (r) aileleri, vCPU başına sırasıyla 2, 4 ve 8 GB bellek sağlar. İsteğe bağlı (on-demand) fiyatlandırmada her sunucu tipi için saatlik sabit bir ücret ödenir [11].

Bu yapı, en ucuz sunucu karışımının talebin işlemci ve bellek bileşenleri arasındaki orana bağlı olmasına yol açar. Örneğin bellek ağırlıklı bir talebin hesaplama optimize

sunucularla karşılanması, kullanılmayan işlemci kapasitesi için ödeme yapılması anlamına gelir. Çalışmada kullanılan üç sunucu tipi ve Ekim 2026 fiyatları Tablo 3.3'te verilmiştir. Rezerve ve spot fiyatlandırma seçenekleri çalışmanın ilk aşamasında kapsam dışında bırakılmıştır.

1.2.3 Hizmet Düzeyi Anlaşması ve Eksik Kapasite

Hizmet düzeyi anlaşması (SLA), hizmet sağlayıcının kullanıcıya taahhüt ettiği erişilebilirlik ve yanıt süresi gibi hedefleri tanımlar [7]. Ayrılan kapasitenin talebin altında kaldığı zaman dilimlerinde istekler kuyrukta bekler veya reddedilir; bu durum SLA hedeflerinin ihlal edilmesi riskini doğurur.

Bu çalışmada yanıt süresi gibi uygulama düzeyindeki ölçütler modellenmemiş, SLA ihlali riski kapasite düzeyinde iki ölçütle temsil edilmiştir. Bunlardan ilki, ayrılan işlemci veya bellek kapasitesinin gerçekleşen tepe talebin altında kaldığı saatlerin oranıdır (eksik kapasiteli saat oranı). İkincisi ise kapasitenin karşılayamadığı talep miktarıdır (karşılanmayan talep). Bu ölçütlerin tanımları Bölüm 3.7'de verilmiştir.

1.3 Çalışmanın Amacı ve Kapsamı

Bu çalışmanın amacı, bir Kubernetes düğüm havuzunun işlemci ve bellek talebini bir gün öncesinden tahmin eden ve bu tahmine göre maliyeti en küçük sunucu planını tamsayılı doğrusal programlama ile belirleyen bir kapasite planlama sistemi tasarlamak ve bu sistemi gerçek iş yükü izleri üzerinde tepkisel ölçekleme yöntemleriyle karşılaştırmaktır. Bu amaç doğrultusunda şu sorulara yanıt aranmaktadır:

(i) Saatlik tepe talep, istatistiksel ve derin öğrenme modelleriyle hangi doğrulukla tahmin edilebilir?

(ii) Tahmin hataları, ILP ile üretilen planın maliyetine ve eksik kapasitesine nasıl yansır?

(iii) Tamsayı kısıtlarının gevşetilip sonucun yukarı yuvarlanması, ILP çözümüne göre ne kadar ek maliyet yaratır?

(iv) Proaktif plan ile tepkisel bir tamponun birleştirilmesi, beklenmeyen talep değişimlerinde nasıl bir maliyet–hizmet dengesi sağlar?

Çalışmada iki gerçek veri kümesi kullanılmıştır: Bitbrains GWA-T-12 [12] ve Alibaba cluster-trace-v2018 [13]. Bu veri kümeleri sanal makine ve fiziksel makine düzeyinde ölçümler içerdiğinden, makinelerin oluşturduğu havuzların toplam talebi bir Kubernetes düğüm havuzunun toplam kaynak talebi için vekil (proxy) olarak kullanılmıştır. Planlama, AWS EC2'nin üç sunucu tipi üzerinden ve yalnızca işlemci ile bellek kaynakları için yapılmıştır.

Pod'ların düğümlere yerleştirilmesi ve bunun yol açtığı parçalanma ile ağ, disk ve GPU gibi diğer kaynaklar çalışmanın kapsamı dışında tutulmuştur. Rezerve ve spot

fiyatlandırma ile talep belirsizliğini doğrudan modele katan yaklaşımlar ise ilk aşamada kapsam dışında bırakılmış, sonraki aşamalarda ele alınması planlanmıştır (Bölüm 5).

1.4 Önerilen Yaklaşım ve Yöntem Seçim Gerekçeleri

Önerilen sistem üç aşamadan oluşur (Şekil 3.1). İlk aşamada, havuz talebinin sonraki 24 saatteki saatlik tepe değerleri zaman serisi modelleriyle tahmin edilir. İkinci aşamada bu tahminler bir ILP modeline girdi olarak verilir ve günün her saati için hangi tipten kaç sunucunun çalıştırılacağı belirlenir. Üçüncü aşamada plana, gerçek zamanlı gözlemlere göre büyüyüp küçülen tepkisel bir tampon eklenir. Planlar yalnızca tahmine göre yapılacak, başarıları ise gerçekleşen talep üzerinden ölçülecektir.

Tahmin aşamasında iki model ailesi kullanılmıştır. SARIMA modelleri, saatlik verideki günlük mevsimselliği açıkça modelleyebilmeleri, yorumlanabilir olmaları ve bulut iş yükü tahmininde yaygın biçimde kullanılmaları nedeniyle seçilmiştir [14, 15]. LSTM ağları ise doğrusal olmayan ilişkileri öğrenebildikleri ve iş yükü tahmininde başarılı sonuçlar verdikleri için tercih edilmiştir [16, 17]. Karmaşık modellerin basit yöntemleri her zaman geçemediğini gösteren çalışmalar [18] dikkate alınarak her iki model, mevsimsel naif ve son değer gibi basit taban modellerle karşılaştırılacaktır.

Planlama aşamasında sunucu sayıları tamsayı olduğundan problem doğal olarak bir tamsayılı programlama problemidir. Tamsayı kısıtları gevşetilerek elde edilen doğrusal programın çözülüp sonucun yukarı yuvarlanması uygun bir plan verir; ancak bu planın en düşük maliyetli olacağının bir güvencesi yoktur [19]. Ayrıca bir sunucunun açılmasının bir maliyeti olduğu ve açılan sunucunun hemen kapatılmasının istenmediği durumlarda saatler birbirinden bağımsız olarak çözülemez. Bu yapı, elektrik sistemlerinde jeneratörlerin devreye alınma planlamasını (unit commitment) ele alan modellere benzer [20]; veri merkezleri için önerilen açma–kapama maliyetli dinamik kapasite modelleri de benzer bir yapıya sahiptir [21]. Bu nedenle çalışmada açma maliyeti ve minimum çalışma süresi kısıtları içeren çok dönemli bir ILP modeli kurulmuştur. Model, lisans gerektirmeyen Google OR-Tools kütüphanesi [22] aracılığıyla SCIP çözücüsüyle [23] çözülmüştür.

Yalnızca tahmine dayanan bir plan, tahmin hatalarına karşı kırılgandır. Tahmine dayalı ve tepkisel ölçeklemenin birlikte kullanılması literatürde daha önce önerilmiş bir yaklaşımdır [24]. Bu çalışmada da ILP planının üzerine, HPA'ya benzer bir eşik kuralıyla yönetilen bir tampon eklenecek ve bu hibrit strateji hem yalnızca tahmine dayalı ILP ile hem de yalnızca tepkisel yöntemlerle karşılaştırılacaktır. Karşılaştırmalarda, gerçek talebi önceden bildiği varsayılan bir kahin (oracle) ILP planı alt sınır olarak kullanılacak; stratejilerin bu alt sınıra göre ek maliyeti (regret), eksik kapasite ölçütleriyle birlikte raporlanacaktır.

1.5 Çalışmanın Özgün Değeri

Çalışmanın hedeflenen katkıları şöyle özetlenebilir:

(i) Talep tahmini, tamsayılı optimizasyon ve tepkisel ölçekleme uçtan uca tek bir sistemde birleştirilmekte; tahmin hatasının karar maliyetine etkisi aynı deney düzeneğinde ölçülecektir.

(ii) Planlama talebi, havuz toplamının saatlik tepesi olarak tanımlanmıştır. Makine bazında hesaplanan tepelerin toplanması eş zamanlı olmayan zirveleri birleştirerek talebi olduğundan yüksek gösterdiğinden, önce havuz toplamı alınmış, tepe değer bu toplam üzerinden hesaplanmıştır.

(iii) Saatleri birbirinden bağımsız ele alan bir model yerine açma maliyeti ve minimum çalışma süresi içeren çok dönemli bir ILP modeli kullanılmış; LP gevşetmesi ve yuvarlama ile elde edilen çözümün ILP çözümüne göre ek maliyeti deneysel olarak ölçülecektir.

(iv) Hibrit stratejide, HPA'nın varsayılan eşiklerinin güvenlik paylı bir ILP planıyla birlikte kullanıldığında tamponun neredeyse sürekli devrede kalmasına yol açacağı gösterilmiş (Bölüm 3.6); bu nedenle plandaki güvenlik payıyla uyumlu eşikler tanımlanmıştır.

(v) Değerlendirme, iki bağımsız gerçek veri kümesi ve her birinde birden çok havuz üzerinde yapılacak; Alibaba kümesindeki bellek varsayımı için duyarlılık analizi uygulanacak ve gerçek veride bulunmayan ani talep değişimleri kontrollü sentetik senaryolarla incelenecektir.

1.6 Raporun Düzeni

Bu rapor, çalışmanın ilk beş haftasını kapsayan birinci ara rapordur ve raporun geri kalanı şöyle düzenlenmiştir. Bölüm 2'de bulut iş yükü tahmini, kapasite planlamasında tamsayılı programlama ve otomatik ölçekleme alanlarındaki çalışmalar incelenmiştir. Bölüm 3'te kullanılan veri kümeleri, veri ön işleme adımları, tahmin modelleri, ILP modeli, karşılaştırma stratejileri ve değerlendirme ölçütleri ayrıntılı olarak anlatılmıştır. Bölüm 4'te ilk beş haftada tamamlanan çalışmalar ile veri kümelerinin keşifsel analizine ve ILP modelinin ilk uygulamasına ait ön sonuçlar verilmiştir. Bölüm 5'te sonraki aşamalar için çalışma planı sunulmuştur. Proje yazılımının yapısı ve ILP modelinin kodundan bir bölüm Ek A'da yer almaktadır.

2. LİTERATÜR TARAMASI

Bu bölümde, çalışmanın dayandığı üç alandaki önceki çalışmalar incelenmiştir: bulut iş yüklerinin zaman serisi yöntemleriyle tahmini, kapasite planlamasında matematiksel programlama ve otomatik ölçekleme yaklaşımları. Bölümün sonunda incelenen çalışmalar bu çalışmanın bileşenleriyle karşılaştırılmış ve çalışmanın literatürdeki konumu belirlenmiştir.

2.1 İstatistiksel Yöntemlerle Bulut İş Yükü Tahmini

Zaman serisi tahmininde en yaygın istatistiksel çerçeve, Box ve Jenkins tarafından sistemleştirilen ARIMA model ailesidir [14]. Bu çerçevede seri, gerekirse fark alınarak durağan hâle getirilir ve geçmiş değerlerle geçmiş hataların doğrusal bir bileşimi olarak modellenir. Mevsimsel ARIMA (SARIMA) modelleri, buna belirli bir periyotla tekrar eden bileşenleri ekler. Saatlik bulut iş yüklerinde gün içi kullanım döngüsü belirgin olduğundan, 24 saatlik mevsim uzunluğuna sahip SARIMA modelleri bu tür veriler için doğal bir seçenektir. Model derecelerinin seçimi, durağanlık testleri ve bilgi ölçütleri gibi standart araçlarla yapılabilir [25].

Calheiros ve arkadaşları, bir hizmet olarak yazılım (SaaS) sağlayıcısına gelen web isteklerini ARIMA modeliyle tahmin etmiş ve bu tahmine göre sanal makine sayısını önceden ayarlayan bir yaklaşım önermiştir [15]. Gerçek web sunucusu istek izleri üzerinde yapılan benzetimlerde ortalama tahmin doğruluğunun %91'e ulaştığı raporlanmış ve tahmin hatasının kaynak kullanımına ve hizmet kalitesine etkisi incelenmiştir. Roy ve arkadaşları, iş yükünü ikinci dereceden bir ARMA filtresiyle tahmin etmiş ve bu tahmini, kaynak maliyeti ile hizmet düzeyi ihlali arasındaki dengeyi gözeten ileriye bakışlı (receding horizon) bir denetleyicide kullanmıştır [6]. Gong ve arkadaşlarının önerdiği PRESS sistemi ise tekrarlayan kullanım örüntülerini sinyal işleme yöntemleriyle, tekrarlamayan örüntüleri ise bir Markov zinciriyle tahmin etmektedir [26]. PRESS'in dikkat çekici bir yönü, eksik tahminin yol açacağı hizmet kayıplarını önlemek için tahmin edilen değeri %5–10 oranında artırmasıdır; yazarlar bu payın eksik tahmin hatalarını neredeyse tamamen ortadan kaldırdığını raporlamıştır. Bu fikir, bu çalışmadaki güvenlik payı α ile aynı amaca hizmet eder.

Bu çalışmaların ortak yönü, tahmin başarısının çoğunlukla ortalama hata ölçütleriyle değerlendirilmesidir. Oysa kapasite planlamasında tahminin talebin altında mı üstünde mi kaldığı simetrik değildir: eksik tahmin hizmet kaybına, fazla tahmin ise yalnızca ek maliyete yol açar. Bu nedenle bu çalışmada ortalama hata ölçütlerinin yanında eksik tahmin oranı da raporlanacak, tahminlerin karar üzerindeki etkisi ayrıca ölçülecektir.

2.2 Derin Öğrenme ile İş Yükü Tahmini

Tekrarlayan sinir ağları (RNN), dizisel verideki zamansal bağımlılıkları öğrenebilen modellerdir; ancak uzun dizilerde gradyanların sönümlenmesi nedeniyle eğitilmeleri güçtür. Hochreiter ve Schmidhuber tarafından önerilen uzun kısa süreli bellek (LSTM) ağları, bilginin hücre durumunda ne kadar tutulacağını denetleyen kapı mekanizmalarıyla

bu sorunu hafifletir [16]. Bu özellik, LSTM'i gün içi döngüler ve ani değişimler içeren iş yükü serileri için çekici kılmıştır.

Kumar ve arkadaşları, bulut veri merkezlerinde iş yükü tahmini için LSTM tabanlı bir model önermiş ve modeli gerçek iş yükü izleri üzerinde değerlendirmiştir [17]. Konteyner ortamlarına yönelik çalışmalarda da benzer bir eğilim görülmektedir. Imdoukh ve arkadaşları, konteyner tabanlı uygulamalar için LSTM tahminine dayalı bir otomatik ölçekleyici önermiş; LSTM'in ARIMA kadar doğru tahmin ürettiğini, ancak yaklaşık 600 kat daha hızlı çalıştığını raporlamıştır [27]. Toka ve arkadaşları Kubernetes uç (edge) kümeleri için makine öğrenmesine dayalı bir ölçekleme yönetimi yaklaşımı geliştirmiştir [28]. Patil ve arkadaşları ise LSTM tabanlı yatay ölçeklemeyi, pod'ların düğümlere atanmasını belirleyen bir karışık tamsayılı doğrusal programlama modeliyle birleştirmiş ve bu yaklaşımın yanıt süresi ve iş hacmi açısından Kubernetes'in varsayılan yöntemlerinden daha iyi sonuç verdiğini raporlamıştır [29].

Öte yandan derin öğrenme modellerinin her durumda basit yöntemlerden üstün olmadığı da gösterilmiştir. Zeng ve arkadaşları, uzun dönemli zaman serisi tahmini için önerilen dönüştürücü (Transformer) tabanlı modellerin, yaygın kullanılan veri kümelerinde tek katmanlı basit doğrusal modeller tarafından geride bırakıldığını raporlamıştır [18]. Bu bulgu, derin öğrenme modellerinin mutlaka güçlü ve basit taban modellerle birlikte değerlendirilmesi gerektiğini göstermektedir. Bu çalışmada LSTM, bu nedenle hem SARIMA hem de mevsimsel naif ve son değer modelleriyle aynı koşullarda karşılaştırılacaktır.

2.3 Kapasite Planlamasında Tamsayılı Programlama

Sunucu sayılarının tamsayı olması, kapasite planlama problemlerinin matematiksel programlama ile ele alınmasında belirleyici bir özelliktir. Tamsayılı programlama problemleri genellikle doğrusal gevşetmenin dal-sınır (branch-and-bound) ve kesme düzlemi yöntemleriyle birleştirilmesiyle çözülür; gevşetilmiş problemin çözümü bir alt sınır verir, ancak bu çözümün yuvarlanmasıyla elde edilen tamsayı çözümün en iyi çözüm olması garanti değildir [19].

Speitkamp ve Bichler, sanal makinelerin fiziksel sunuculara yerleştirilmesini (sunucu konsolidasyonu), iş yüklerinin gün içindeki değişimini dikkate alan bir tamsayılı programlama problemi olarak modellemiş; büyük örnekler için doğrusal gevşetmeye dayanan sezgisel yöntemler önermiştir [30]. Beloglazov ve Buyya ise sanal makinelerin dinamik konsolidasyonu için enerji tüketimi ile performans arasında denge kuran çevrimiçi algoritmalar ve uyarlanabilir sezgiseller geliştirmiştir [31]. Bu çalışmalar bir sanal makine kümesinin sunuculara nasıl yerleştirileceği sorusuna odaklanır; bu çalışmada ise yerleştirme yerine, toplam talebi karşılayacak sunucu filosunun tip ve sayı bakımından nasıl oluşturulacağı ele alınmıştır.

Zaman içinde değişen talep altında kapasite planlamasında önemli bir unsur, sunucuların açılıp kapatılmasının bir maliyeti olmasıdır. Lin ve arkadaşları, veri merkezinde çalışan sunucu sayısının zamana göre ayarlanmasını açma–kapama maliyetini içeren bir

optimizasyon problemi olarak ele almış ve bu problem için çevrimiçi bir algoritma önermiştir [21]. Benzer bir yapı, elektrik sistemlerindeki devreye alma (unit commitment) probleminde de bulunur. Bu problemde jeneratörlerin hangi saatlerde çalışacağı; devreye alma maliyetleri, minimum çalışma ve minimum duruş süreleri dikkate alınarak karışık tamsayılı doğrusal programlama ile belirlenir. Carrión ve Arroyo, bu problem için ikili değişken sayısını azaltan ve hesaplama açısından verimli bir formülasyon önermiştir [20]. Bu çalışmadaki çok dönemli ILP modeli bu formülasyonlardan esinlenmiştir: sunucu açma maliyeti ve minimum çalışma süresi kısıtları, jeneratörlerin devreye alma maliyetine ve minimum çalışma süresine karşılık gelir.

Bu tür modellerin çözümünde ticari çözücülerin yanında açık kaynaklı çözücüler de kullanılabilir. SCIP, kısıt programlama ile karışık tamsayılı programlama tekniklerini dal-kesme çerçevesinde birleştiren, akademik kullanımda yaygın bir çözücüdür [23].

2.4 Proaktif, Reaktif ve Hibrit Otomatik Ölçekleme

Lorido-Botran ve arkadaşları, bulut uygulamaları için otomatik ölçekleme yöntemlerini eşik tabanlı kurallar, pekiştirmeli öğrenme, kuyruk teorisi, denetim teorisi ve zaman serisi analizi olarak sınıflandırmış ve bu yöntemleri tepkisel (reactive) ve proaktif olmak üzere iki grupta değerlendirmiştir [5]. Qu ve arkadaşları da web uygulamalarının otomatik ölçeklenmesine yönelik benzer bir sınıflandırma sunmuş ve kaynak tahmini ile ölçekleme kararlarının birlikte ele alınmasının önemini vurgulamıştır [7].

Eşik tabanlı kurallar, basit ve anlaşılır olmaları nedeniyle uygulamada en yaygın yaklaşımdır. Kubernetes HPA'sı da bu türden bir kuraldır ve izlenen ölçütün hedef değere oranına göre pod sayısını değiştirir [4]. Google'ın Autopilot sistemi, kaynak isteklerini geçmiş kullanım pencerelerine ve makine öğrenmesine dayalı öneri modelleriyle otomatik olarak ayarlamakta ve büyük ölçekli bir üretim ortamında ayrılan ile kullanılan kaynak arasındaki farkı azalttığını raporlamaktadır [9].

Gandhi ve arkadaşlarının AutoScale çalışması, sunucuların açılma süresinin (setup time) ölçekleme kararları üzerindeki etkisini incelemiştir [32]. Bu çalışmada, açılması uzun süren sunucuların talep düştüğünde hemen kapatılmaması, bunun yerine bir süre boşta bekletilmesi önerilmiştir. Bu fikir, bu çalışmadaki açma maliyeti ve minimum çalışma süresi kısıtlarının uygulamadaki karşılığıdır.

Tahmine dayalı ve tepkisel yaklaşımların birleştirilmesi de literatürde ele alınmıştır. Urgaonkar ve arkadaşları, çok katmanlı internet uygulamaları için uzun zaman ölçeğinde (saatler, günler) tahmine dayalı, kısa zaman ölçeğinde ise beklenmeyen ani yük artışlarına karşı tepkisel kapasite ayıran iki düzeyli bir yöntem önermiştir [24]. Bu çalışmadaki hibrit strateji aynı ilkeye dayanır: günlük kapasite planı tahmine göre ILP ile belirlenir, saatlik sapmalar ise tepkisel bir tamponla karşılanır.

2.5 Değerlendirme ve Araştırma Boşluğu

İncelenen çalışmaların bu çalışmanın bileşenleri açısından karşılaştırması Tablo 2.1'de verilmiştir. Tabloda her çalışmanın talep tahmini kullanıp kullanmadığı, kapasite kararını tamsayılı optimizasyonla verip vermediği, açma maliyeti veya dönemler arası bağ içerip içermediği ve tepkisel bir bileşen barındırıp barındırmadığı gösterilmiştir.

Tablo 2.1 İncelenen çalışmaların bu çalışmanın bileşenleri açısından karşılaştırılması

Çalışma	Talep tahmini	Tamsayılı optimizasyon	Açma maliyeti / çok dönem	Tepkisel bileşen
Calheiros vd. [15]	ARIMA	Hayır	Hayır	Hayır
Roy vd. [6]	ARMA	Hayır	Hayır	Hayır
Kumar vd. [17]	LSTM	Hayır	Hayır	Hayır
Speitkamp ve Bichler [30]	Hayır (geçmiş profiller)	Evet	Hayır	Hayır
Lin vd. [21]	Hayır	Hayır	Evet	Evet (çevrimiçi)
Gandhi vd. [32]	Hayır	Hayır	Evet	Evet
Urgaonkar vd. [24]	Evet	Hayır	Hayır	Evet
Bu çalışma	SARIMA, LSTM	Evet (ILP)	Evet	Evet

Tablo 2.1'den görüldüğü gibi, tahmin odaklı çalışmalar kapasite kararını çoğunlukla basit bir dönüşümle, örneğin tahmin edilen talebin tek tip sunucu kapasitesine bölünmesiyle verir ve tahmin başarısını ortalama hata ölçütleriyle değerlendirir. Optimizasyon odaklı çalışmalar ise talebin bilindiğini veya geçmiş profillerden elde edilebileceğini varsayar. Tahmini ve tamsayılı programlamayı Kubernetes bağlamında birleştiren Patil ve arkadaşlarının çalışmasında [29] amaç, pod yerleşimi üzerinden yanıt süresi ve iş hacmidir; sunucu tiplerinin maliyeti ve açma maliyeti modelde yer almaz. Açma maliyetini ve dönemler arası bağı dikkate alan çalışmalar genellikle tek tip sunucu üzerinden çevrimiçi kararlara odaklanır.

Bu çalışma, söz konusu bileşenleri tek bir sistemde birleştirmeyi amaçlamaktadır: talep istatistiksel ve derin öğrenme modelleriyle tahmin edilmekte, tahminler farklı işlemci–bellek oranlarına sahip sunucu tiplerini, açma maliyetini ve minimum çalışma süresini içeren çok dönemli bir ILP modeline girdi olarak verilmekte ve elde edilen plan tepkisel bir tamponla desteklenmektedir. Ayrıca tahmin hatası, karar maliyeti (regret) ve eksik kapasite aynı deney düzeneğinde, iki gerçek veri kümesi üzerinde birlikte ölçülecektir. Böylece tahmin doğruluğundaki farkların maliyet ve hizmet sürekliliği açısından ne anlama geldiği doğrudan gösterilebilecektir.

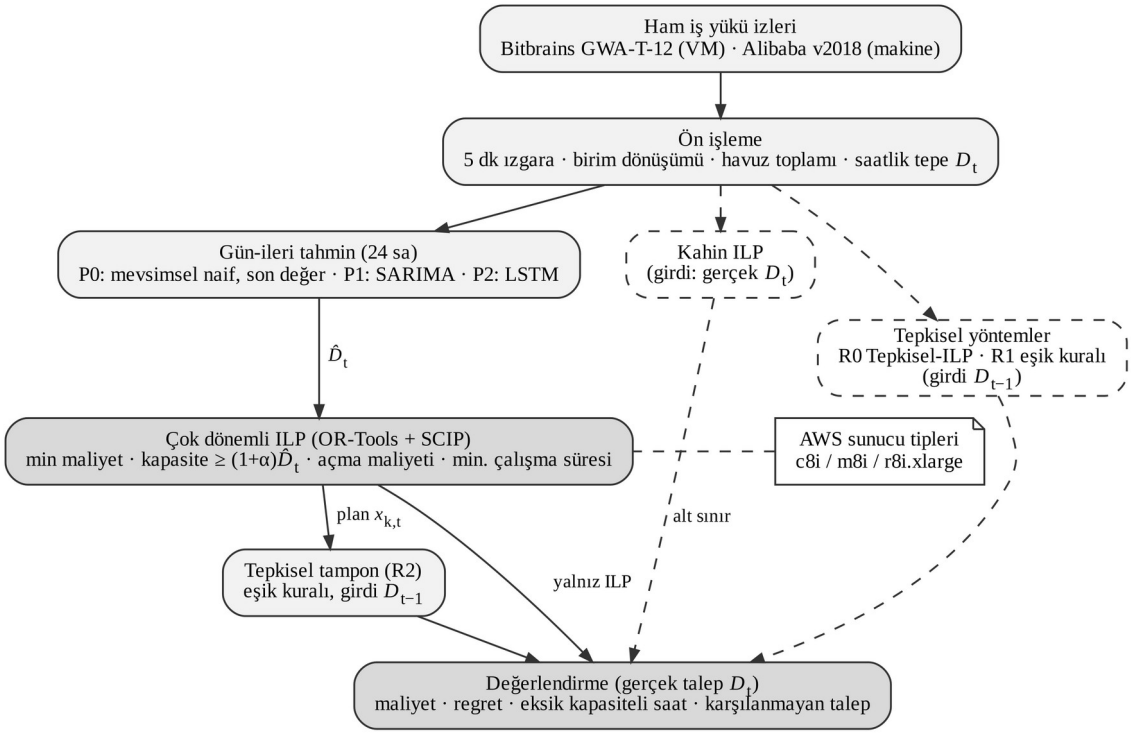
3. METODOLOJİ VE MATEMATİKSEL MODEL

Bu bölümde önerilen kapasite planlama sisteminin bileşenleri ayrıntılı olarak anlatılmıştır. Önce sistemin genel mimarisi tanıtılmış, ardından kullanılan veri kümeleri ve ön işleme adımları açıklanmıştır. Daha sonra tahmin modelleri, çok dönemli ILP modeli, karşılaştırma stratejileri, değerlendirme ölçütleri ve duyarlılık ile sentetik senaryo deneylerinin tasarımı verilmiştir. Bölümde kullanılan tüm sayısal parametreler projenin yapılandırma dosyalarından alınmıştır.

3.1 Genel Sistem Mimarisi

Önerilen sistemin genel yapısı Şekil 3.1'de görüldüğü gibidir. Ham iş yükü izleri önce ortak bir zaman ızgarasına hizalanmış, birimleri dönüştürülmüş ve havuz düzeyinde saatlik tepe talep serilerine çevrilmiştir. Her günün başında bu seriler kullanılarak sonraki 24 saatin işlemci ve bellek tepe talepleri tahmin edilir ve tahminler çok dönemli ILP modeline girdi olarak verilir. ILP modeli, günün her saati için hangi sunucu tipinden kaç adet çalıştırılacağını belirler. Hibrit stratejide bu plana, bir önceki saatin gerçekleşen talebine göre büyüyüp küçülen tepkisel bir tampon eklenir.

Sistemin temel ilkesi, planın yalnızca tahmine göre yapılması, başarının ise gerçekleşen talep üzerinden ölçülmesidir. ILP modeli gerçek talebi hiçbir zaman görmez. Karşılaştırma için aynı model gerçek talep ile de çözülecek (kahin ILP) ve bu çözüm, mükemmel öngörü altında ulaşılabilecek en düşük maliyet olarak alt sınır kabul edilecektir. Tahmin kullanmayan tepkisel yöntemler (R0 ve R1) de aynı gerçek talep üzerinde değerlendirilecektir.



Şekil 3.1 Önerilen kapasite planlama sisteminin genel mimarisi

3.2 Veri Kümeleri

Çalışmada farklı nitelikte iki gerçek iş yükü izi kullanılmıştır. Veri kümelerinin ve bunlardan oluşturulan havuzların temel özellikleri Tablo 3.1'de özetlenmiştir.

3.2.1 Bitbrains GWA-T-12 Veri Kümesi

Bitbrains GWA-T-12 veri kümesi, iş açısından kritik kurumsal uygulamalara hizmet veren bir veri merkezindeki sanal makinelerin kaynak kullanım ölçümlerini içerir [12]. Veri kümesi Bitbrains IT Services Inc. tarafından yayımlanmıştır. Veri kümesi iki bölümden oluşur: hızlı depolama birimine bağlı 1.250 sanal makineden oluşan fastStorage ve 500 sanal makineden oluşan Rnd. Her sanal makine için ayrı bir dosyada yaklaşık 5 dakikalık aralıklarla işlemci çekirdek sayısı, tanımlı işlemci kapasitesi (MHz), işlemci kullanımı (MHz) ve bellek kullanımı (KB) bilgileri yer alır.

fastStorage tek bir aylık klasörden oluşur ve 12.08.2013–11.09.2013 dönemini kapsar. Rnd ise üç aylık klasöre bölünmüştür; bu klasörlerin farklı dönemlerdeki farklı makineleri içerebileceği göz önünde bulundurularak yalnızca Ağustos 2013 klasörü (31.07.2013–30.08.2013) kullanılmış ve aylar birleştirilmemiştir. Her iki havuzda da 30 günlük (720 saatlik) seri elde edilmiştir. fastStorage ve Rnd, farklı iş yükü karakterlerini temsil ettikleri için hiçbir aşamada birleştirilmemiş, ayrı havuzlar olarak ele alınmıştır.

3.2.2 Alibaba Cluster-Trace-v2018 Veri Kümesi

Alibaba cluster-trace-v2018 veri kümesi, çevrimiçi servislerle toplu işlerin aynı makinelerde birlikte çalıştırıldığı bir üretim kümesinden alınmış 8 günlük izleri içerir [13]. Bu çalışmada makine düzeyindeki kullanım ölçümlerini içeren machine_usage tablosu ile makine özelliklerini içeren machine_meta tablosu kullanılmıştır. machine_usage tablosu 4.023 makine için yaklaşık 10 saniyelik aralıklarla alınmış 246.934.820 ölçüm içerir. Tüm makinelerin işlemci sayısı (cpu_num) 96'dır. İşlemci ve bellek kullanımı ise makinenin kapasitesine oranla yüzde olarak verilmiştir. machine_meta tablosundaki bellek miktarı normalize edilmiş olup fiziksel bellek miktarı veri kümesinde yer almaz; bu nedenle bellek için Bölüm 3.3.3'te açıklanan bir varsayım gerekmiştir.

Veri kalitesi incelendiğinde, 8 günün tamamında makine ömrü içindeki uzun ölçüm boşluklarının oranı %8,76 bulunmuştur. Bu oran, çalışmada belirlenen %1 eşiğinin oldukça üzerindedir. Boşlukların iz başında yoğunlaştığı görülmüştür: ilk yaklaşık 39 saatte makinelerin yalnızca %58–77'si ölçüm göndermiş, 39–44. saatler arasında ise küme genelinde ölçüm kesintisi yaşanmıştır. Bu nedenle ilk iki gün atılmış ve analiz 2–7. günler (144 saat) üzerinde yapılmıştır. Bu pencerede uzun boşluk oranı %0,15'e düşmüş ve analize 4.022 makine girmiştir. İz yalnızca 8 gün sürdüğü ve 6 günü kullanılabildiği için haftalık mevsimsellik öğrenilememiş ve test dönemi tek günle sınırlı kalmıştır. Bu durum çalışmanın bilinen bir sınırlılığıdır ve Alibaba sonuçlarının yorumlanmasında dikkate alınacaktır.

Tablo 3.1 Veri kümelerinin ve havuzların özellikleri

Özellik	Bitbrains fastStorage	Bitbrains Rnd	Alibaba v2018
Ölçüm düzeyi	Sanal makine	Sanal makine	Fiziksel makine
Birim sayısı	1.250	500	4.023 (pencerede 4.022)
Dönem	12.08–11.09.2013	31.07–30.08.2013	8 gün (göreli zaman)
Ham örnekleme aralığı	≈ 5 dk	≈ 5 dk	≈ 10 s
İşlemci ölçüsü	Kullanım ve kapasite (MHz), çekirdek sayısı	Kullanım ve kapasite (MHz), çekirdek sayısı	Kullanım yüzdesi, cpu_num = 96
Bellek ölçüsü	Kullanım (KB)	Kullanım (KB)	Kullanım yüzdesi (fiziksel GB yok)
Analiz penceresi	30 gün (720 sa)	30 gün (720 sa)	2–7. günler (144 sa)
Bölme (eğitim / doğrulama / test, gün)	20 / 4 / 6	20 / 4 / 6	4 / 1 / 1
Havuzlar	Doğal havuz + 5 alt havuz (250 VM)	Doğal havuz	8 havuz (500 makine) × r ∈ {2, 4, 8}

3.3 Veri Ön İşleme

Ön işleme adımları, ham ölçümleri planlamada kullanılacak havuz düzeyindeki saatlik tepe talep serilerine dönüştürmek amacıyla aşağıdaki sırayla uygulanmıştır.

3.3.1 Zaman Hizalama ve Birim Dönüşümleri

Farklı makinelerin ölçümleri farklı anlarda alındığından, tüm seriler 5 dakikalık ortak bir zaman ızgarasına hizalanmıştır. Her ölçüm, zaman damgasının düştüğü 5 dakikalık hücreye atanmış; aynı hücreye birden fazla ölçüm düştüğünde bunların ortalaması alınmıştır. Zaman damgasının birimi sütun adına güvenilmeden, ardışık ölçümler arasındaki farkların medyanından belirlenmiştir (Bitbrains için 300 s). Alibaba'da mutlak tarih bilinmediğinden zaman, iz başlangıcına göre göreli olarak ele alınmıştır.

Bitbrains'te işlemci talebi, her sanal makinenin kendi tanımlı kapasitesine göre çekirdek eşdeğerine çevrilmiştir. v sanal makinesinin τ hücresindeki işlemci kullanımı $U_{v,\tau}$ (MHz), çekirdek sayısı $n_{v,\tau}$ ve tanımlı kapasitesi $C^{tan}_{v,\tau}$ (MHz) ile gösterildiğinde işlemci talebi Eşitlik 3.1'deki gibi, KB cinsinden bellek kullanımı $B_{v,\tau}$ ise Eşitlik 3.2'deki gibi GiB'a çevrilmiştir.

$$c_{v,\tau} = U_{v,\tau} \frac{n_{v,\tau}}{C^{tan}_{v,\tau}} \tag{3.1}$$

$$m_{v,\tau} = \frac{B_{v,\tau}}{1024^2} \tag{3.2}$$

Çekirdek sayısının sıfır olduğu kayıtlar geçersiz kabul edilmiştir; bu kayıtlar fastStorage'da tüm satırların %14,2'sini, Rnd'de %1,4'ünü oluşturur ve neredeyse tamamı sanal makinenin henüz başlamadığı veya kapatıldığı dönemlere aittir. Kullanımın tanımlı kapasiteyi aştığı kayıtlar (fastStorage'da %2,95, Rnd'de %2,19) ise kırpılmamıştır. Bu kayıtlar, sanal makinenin kısa süreli olarak kendisine ayrılandan fazla işlemci kullanabildiği gerçek talep artışlarını yansıttığından korunmuştur. Tam kopya satırlar veri kümesinden çıkarılmıştır.

Alibaba'da j makinesinin işlemci talebi, kullanım yüzdesi $p^{cpu}_{j,\tau}$ ve işlemci sayısı n_j ile Eşitlik 3.3'teki gibi hesaplanmıştır. [0, 100] aralığı dışındaki yüzde değerleri geçersiz sayılmıştır.

$$c_{j,\tau} = \frac{p^{cpu}_{j,\tau}}{100} n_j \tag{3.3}$$

3.3.2 Eksik Veri ve Yaşam Süresi Kuralları

Bir makinenin yaşam süresi, ilk ve son geçerli ölçümü arasındaki dönem olarak tanımlanmıştır. Yaşam süresi dışında makinenin talebi sıfır kabul edilmiştir; çünkü bu dönemlerde makine henüz oluşturulmamış veya kaldırılmıştır. Yaşam süresi içindeki en fazla 3 hücrelik (15 dakika) boşluklar doğrusal enterpolasyonla doldurulmuştur. Daha uzun boşluklar doldurulmamış, havuz toplamında sıfır olarak sayılmış ve bu hücrelerin oranı ayrıca izlenmiştir. Bu oran fastStorage'da %0,02, Rnd'de %0,09 ve Alibaba analiz penceresinde %0,15 olup her üç durumda da %1 eşiğinin altında kalmıştır.

3.3.3 Referans Bellek Varsayımı

Alibaba veri kümesinde bellek kullanımı yalnızca yüzde olarak verildiğinden ve fiziksel bellek miktarı bilinmediğinden, her makine için bir referans bellek miktarı M_{ref} tanımlanmıştır. Referans bellek, makinenin işlemci sayısı ile çekirdek başına bellek katsayısı r'nin çarpımıdır ve bellek talebi Eşitlik 3.4'teki gibi hesaplanmıştır.

$$m_{j,\tau} = \frac{p_{j,\tau}^{mem}}{100} M_{ref}, M_{ref} = n_j r \tag{3.4}$$

r katsayısı için merkez değer 4 GB/çekirdek alınmış ve duyarlılık analizi için $r \in \{2, 4, 8\}$ değerleri kullanılmıştır. Bu değerler, AWS'nin hesaplama optimize, genel amaçlı ve bellek optimize ailelerinin vCPU başına bellek oranlarına karşılık gelir. 96 işlemcili bir makine için referans bellek sırasıyla 192, 384 ve 768 GB olur. Bu varsayım yalnızca bellek talebini etkiler; bu nedenle tüm Alibaba deneyleri üç r değeri için ayrı ayrı tekrarlanacaktır. Alibaba bellek sonuçları bu nedenle M_{ref} varsayımına bağlı olarak yorumlanmalıdır.

3.3.4 Havuz Toplamı ve Saatlik Tepe Talep

Planlama talebinin hesaplanmasında işlemlerin sırası belirleyicidir. Önce her 5 dakikalık τ hücresinde havuzdaki tüm makinelerin talebi toplanmış (Eşitlik 3.5), ardından her h saati için bu toplamın 12 hücre üzerindeki en büyük değeri saatlik tepe talep olarak alınmıştır (Eşitlik 3.6). Bellek talebi de aynı biçimde hesaplanmıştır. Saatlik ortalama talep yalnızca karşılaştırma amacıyla ayrıca saklanmıştır.

$$D_{\tau}^{5} = \sum_{v \in V} c_{v,\tau} \tag{3.5}$$

$$D_h = \max_{\tau \in h} D_\tau^5, \acute{D}_h = \frac{1}{12} \sum_{\tau \in h} D_\tau^5 \quad (3.6)$$

Tepelerin makine bazında alınıp toplanması bu sıranın tersi olur ve Eşitlik 3.7'de görüldüğü gibi talebi olduğundan yüksek gösterir. Makinelerin tepe anları genellikle çakışmadığından, tepelerin toplamı havuzun gerçekte karşılaması gereken en yüksek talepten büyüktür. Kapasite havuzun toplamı için ayrıldığından, doğru planlama talebi toplamın tepesidir.

$$\max_{\tau \in h} \sum_{v \in V} c_{v,\tau} \leq \sum_{v \in V} \max_{\tau \in h} c_{v,\tau} \quad (3.7)$$

3.3.5 Havuz Tasarımı ve Zaman Bölmesi

Sonuçların tek bir havuza bağlı kalmaması için birden çok havuz oluşturulmuştur. Bitbrains'te fastStorage ve Rnd doğal havuzlarının yanında, fastStorage sanal makineleri sabit bir tohumla (42) rastgele ve ayrık olarak 250'şer makinelik 5 alt havuza bölünmüştür. Alibaba'da analiz penceresindeki makineler aynı tohumla 500'er makinelik 8 ayrık havuza ayrılmış, artan 22 makine hiçbir havuza alınmamıştır. Her Alibaba havuzu üç r değeriyle çalıştırıldığından toplam 24 Alibaba serisi elde edilmiştir. Sonuçlar havuzlar üzerinden ortalama ± standart sapma olarak raporlanacaktır.

Seriler kronolojik olarak, karıştırılmadan eğitim, doğrulama ve test bölümlerine ayrılmıştır (Tablo 3.1). Doğrulama bölümü model seçimi ve erken durdurma için, test bölümü ise yalnızca nihai karşılaştırma için kullanılmıştır. Test dönemindeki hiçbir bilgi model seçiminde, hiperparametre ayarında veya veri ölçeklemede kullanılmamıştır.

3.4 Zaman Serisi Tahmin Modelleri

Bu alt bölümde tahmin görevi tanımlanmış ve kullanılan üç model grubu açıklanmıştır: parametresiz taban modeller (P0), SARIMA (P1) ve LSTM (P2).

3.4.1 Gün-İleri Tahmin Görevi ve İleri Yürüyen Değerlendirme

Her havuz için işlemci ve bellek saatlik tepe talep serileri ayrı ayrı tahmin edilmiştir. Tahmin ufku 24 saattir: her günün başındaki o kökeninde, yalnızca o anından önceki veriler kullanılarak o, o+1, …, o+23 saatlerinin tepe talepleri tahmin edilmiştir. Test dönemi ileri yürüyen (walk-forward) biçimde değerlendirilecektir; yani her test gününün başında o güne kadar gözlenen tüm veriler girdi olarak kullanılacaktır. Model parametreleri ise eğitim ve doğrulama verisiyle bir kez belirlenmiş olup test boyunca sabit tutulacaktır.

3.4.2 Taban Modeller (P0)

İki parametresiz taban model kullanılmıştır. Mevsimsel naif model, her saat için önceki günün aynı saatindeki değeri tahmin olarak kullanır (Eşitlik 3.8). Gün-ileri son değer modeli ise gün başında gözlenen son tepe değeri 24 saat boyunca sabit tutar (Eşitlik 3.9). Bu modeller, daha karmaşık modellerin sağladığı kazancın ölçülebilmesi için referans olarak kullanılmıştır.

$$\hat{y}_{o+h} = y_{o+h-24}, h = 0, 1, \ldots, 23 \tag{3.8}$$

$$\hat{y}_{o+h} = y_{o-1}, h = 0, 1, \ldots, 23 \tag{3.9}$$

3.4.3 SARIMA Modeli (P1)

Mevsim uzunluğu s = 24 saat olan SARIMA(p, d, q)×(P, D, Q)$_s$ modeli Eşitlik 3.10 ile tanımlanır [14]. Burada B geri kaydırma işlecidir (B y_t = y_{t-1}); φ_p ve θ_q mevsimsel olmayan, Φ_P ve Θ_Q mevsimsel otoregresif ve hareketli ortalama çok terimlileridir; ε_t ise beyaz gürültüdür.

$$\phi_p(B)\Phi_P(B^s)(1-B)^d(1-B^s)^D y_t = \theta_q(B)\Theta_Q(B^s)\varepsilon_t \tag{3.10}$$

Model dereceleri her seri (havuz × kaynak) için yalnızca eğitim ve doğrulama verisiyle, aşağıdaki adımlarla seçilmiştir:

(i) Mevsimsel olmayan fark derecesi d, eğitim döneminde ADF [33] ve KPSS [34] testleriyle %5 anlamlılık düzeyinde belirlenmiştir. d = 0 ancak ADF birim kök hipotezini reddettiğinde ve KPSS durağanlık hipotezini reddetmediğinde seçilmiş, aksi hâlde birinci farkta testler tekrarlanmıştır (en fazla d = 1).

(ii) Mevsimsel fark derecesi D, eğitim döneminde STL ayrışımından [35] elde edilen mevsimsellik gücüne (Eşitlik 3.11) göre belirlenmiştir [36]. $F_s \geq 0{,}64$ ise D = 1, değilse D = 0 alınmıştır [25]. Eşitlikte S_t ve R_t sırasıyla STL ayrışımının mevsimsel ve artık bileşenleridir.

(iii) p, q ∈ {0, 1, 2} ve P, Q ∈ {0, 1} değerlerinden oluşan aday ızgarası iki aileye ayrılmıştır: mevsimsel olmayan aile (P = Q = 0) ve mevsimsel aile ((P, Q) ≠ (0, 0)). Her aile içinde, aynı (d, D) değerlerine sahip adaylar eğitim verisine uydurulmuş ve Eşitlik 3.12'deki Akaike bilgi ölçütü (AIC) en küçük olan model seçilmiştir [37]. Burada $\hat{L}$ en büyük olabilirlik değeri, k ise parametre sayısıdır.

(iv) İki ailenin kazananları arasındaki nihai seçim, doğrulama dönemindeki gün-ileri tahminlerin ortalama mutlak hatasına göre yapılmıştır. AIC yalnızca aynı fark

derecesine sahip modeller arasında karşılaştırılabildiğinden, farklı aileler arasında örneklem dışı hata ölçütü tercih edilmiştir.

$$F_s = max\left(0, 1 - \frac{Var(R_t)}{Var(S_t + R_t)}\right) \tag{3.11}$$

$$AIC = -2\ln\hat{L} + 2k \tag{3.12}$$

Seçilen model eğitim ve doğrulama verisiyle yeniden uydurulmuştur. Test döneminde parametreler sabit tutulacak, modelin durumu yeni gözlemlerle yeniden uydurma yapılmadan güncellenecektir. Bu işlemler statsmodels kütüphanesinin SARIMAX uygulamasıyla yapılmıştır.

3.4.4 LSTM Modeli (P2)

LSTM hücresi, x_t girdisi ve bir önceki gizli durum h_{t-1} ile Eşitlik 3.13–3.18'deki gibi çalışır [16]. Burada σ lojistik fonksiyon, ⊙ eleman bazında çarpım, f_t, i_t ve o_t sırasıyla unutma, girdi ve çıktı kapıları, c_t ise hücre durumudur. W, U ve b öğrenilen ağırlık ve sapma parametreleridir.

$$f_t = \sigma\left(W_f x_t + U_f h_{t-1} + b_f\right) \tag{3.13}$$

$$i_t = \sigma\left(W_i x_t + U_i h_{t-1} + b_i\right) \tag{3.14}$$

$$\tilde{c}_t = \tanh\left(W_c x_t + U_c h_{t-1} + b_c\right) \tag{3.15}$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \tag{3.16}$$

$$o_t = \sigma\left(W_o x_t + U_o h_{t-1} + b_o\right) \tag{3.17}$$

$$h_t = o_t \odot \tanh\left(c_t\right) \tag{3.18}$$

Modelin girdisi, tahmin kökeninden önceki bakış penceresindeki 5 dakikalık havuz toplamlarıdır ve işlemci ile bellek olmak üzere iki kanaldan oluşur. Saatlik tepe saat içindeki zirveden belirlendiğinden, girdi olarak saatlik özet yerine 5 dakikalık çözünürlük kullanılmıştır. Bakış penceresi Bitbrains'te 48 saat (576 adım) olarak seçilmiştir. Alibaba'da ise 96 saatlik eğitim döneminde yeterli sayıda eğitim örneği kalması için bu süre 24 saate (288 adım) indirilmiştir; böylece eğitim kökeni sayısı 25'ten 49'a çıkmıştır. Son LSTM katmanının gizli durumu doğrusal bir katmana verilmiş ve sonraki 24 saatin işlemci ve bellek tepe talepleri (48 değer) tek seferde üretilmiştir. Modelin yapısı ve eğitim ayarları Tablo 3.2'de verilmiştir.

Tablo 3.2 LSTM modelinin yapısı ve eğitim ayarları

Ayar	Değer
Girdi	5 dk havuz toplamları, 2 kanal (işlemci, bellek)
Bakış penceresi	Bitbrains 48 sa, Alibaba 24 sa
Çıktı	24 sa × 2 kaynak = 48 değer
Katman sayısı / gizli birim	2 / 64
Dropout	0,1
Kayıp fonksiyonu / eniyileyici	MSE / Adam (öğrenme oranı 0,001)
Yığın boyutu	32
Erken durdurma	Doğrulama kaybı, sabır 30 epoch, en fazla 300 epoch
Ölçekleme	Kanal başına standartlaştırma (yalnız eğitim verisi)
Tohumlar	0, 1, 2 (ortalama ± standart sapma)
Donanım	NVIDIA RTX 3050, karışık duyarlıklı eğitim

Eğitim iki aşamada yapılmıştır. İlk aşamada ölçekleyici yalnızca eğitim verisine uydurulmuş ve model, doğrulama kaybı 30 epoch boyunca iyileşmeyene kadar eğitilmiştir. En iyi doğrulama kaybının elde edildiği epoch sayısı Bitbrains'te 5–59, Alibaba'da 56–300 arasında değişmiştir. İkinci aşamada ölçekleyici eğitim ve doğrulama verisine yeniden uydurulmuş ve model aynı tohumla, ilk aşamada bulunan epoch sayısı kadar eğitilmiştir. Test döneminde ağırlıklar sabit tutulacak, yalnızca girdi penceresi ilerletilecektir. Rastgeleliğin etkisini görmek için her havuzda üç farklı tohum kullanılmıştır. Uygulama PyTorch kütüphanesiyle yapılmıştır.

3.4.5 Tahmin Başarım Ölçütleri

Tahmin başarısı, test dönemindeki N tahmin dilimi üzerinden Eşitlik 3.19–3.22'deki ölçütlerle ölçülecektir. MASE ölçütünün paydası, eğitim döneminde mevsimsel naif modelin örneklem içi ortalama mutlak hatasıdır [38]; MASE < 1, modelin bu tabandan

daha iyi olduğunu gösterir. Eksik tahmin oranı (Eşitlik 3.22), tahminin gerçekleşen değerin altında kaldığı dilimlerin oranıdır. Bu ölçüt, kapasite planlamasında eksik tahminin fazla tahminden daha ciddi sonuçlar doğurması nedeniyle eklenmiştir. Tüm ölçütler işlemci ve bellek için ayrı ayrı hesaplanacaktır.

$$MAE = \frac{1}{N}\sum_{t=1}^{N}\left|y_t - \hat{y}_t\right|, RMSE = \sqrt{\frac{1}{N}\sum_{t=1}^{N}\left(y_t - \hat{y}_t\right)^2} \tag{3.19}$$

$$MAPE = \frac{100}{N}\sum_{t=1}^{N}\frac{\left|y_t - \hat{y}_t\right|}{y_t} \tag{3.20}$$

$$MASE = \frac{MAE}{\frac{1}{n-24}\sum_{t=25}^{n}\left|y_t - y_{t-24}\right|} \tag{3.21}$$

Eşitlik 3.21'de n eğitim dönemindeki saat sayısıdır. Eksik tahmin oranı ise Eşitlik 3.22'deki gösterge fonksiyonuyla tanımlanmıştır.

$$ETO = \frac{1}{N}\sum_{t=1}^{N} 1\left[\hat{y}_t < y_t\right] \tag{3.22}$$

3.5 Kapasite Planlama Modeli

Kapasite planlama modeli, elektrik sistemlerindeki devreye alma (unit commitment) problemine benzer biçimde kurulmuştur [20]. Saatler, sunucu açma maliyetleri ve minimum çalışma süresi kısıtlarıyla birbirine bağlanmıştır; bu nedenle model her saat için bağımsız bir sırt çantası problemi değil, çok dönemli bir tamsayılı programlama problemidir.

3.5.1 Kümeler, Parametreler ve Karar Değişkenleri

K sunucu tipleri kümesini, t ∈ {1, …, H} saatlik zaman dilimlerini göstermektedir. Planlama ufku H = 24 saattir. Modelde kullanılan sunucu tipleri, aynı nesilden (Intel Granite Rapids) ve aynı boyuttan (4 vCPU), ancak farklı GB/vCPU oranlarına sahip üç AWS EC2 tipidir. Bu tipler ve fiyatları Tablo 3.3'te verilmiştir. Fiyatlar, AWS fiyat listesi programlama arayüzünden 29.10.2026 tarihinde çekilmiş ve bu tarihle sabitlenmiştir [11].

Talepteki bir çekirdek bir vCPU'ya eşlenmiştir. Maliyet yalnızca sunucu saat ücretinden oluşur; ağ, depolama ve indirimli fiyatlandırma seçenekleri kapsam dışındadır.

Tablo 3.3 Sunucu tipleri ve özellikleri (AWS EC2 on-demand, Linux, eu-central-1)

Tip	Aile	vCPU	Bellek (GiB)	GB/ vCPU	Saatlik maliyet (USD)
c8i.xlarge	Hesaplama optimize	4	8	2	0,21388
m8i.xlarge	Genel amaçlı	4	16	4	0,25358
r8i.xlarge	Bellek optimize	4	32	8	0,33516

Her $k \in K$ tipi için c_k saatlik maliyeti, C_k vCPU kapasitesini, M_k bellek kapasitesini (GiB), s_k açma maliyetini, d_k kapatma maliyetini ve L_k minimum çalışma süresini (dilim) göstermektedir. α güvenlik payı, ρ^{cpu} ve ρ^{mem} ise kapasitenin kullanılabilir oranıdır. $\hat{D}^{cpu}_t$ ve $\hat{D}^{mem}_t$, t dilimi için tahmin edilen tepe taleplerdir. x^0_k ufuk başında çalışan, $\hat{y}_{k,\tau}$ ($\tau \leq 0$) ise önceki günün son $L_k - 1$ diliminde açılan sunucu sayısıdır.

Modelin karar değişkenleri, t diliminde çalışan k tipi sunucu sayısı $x_{k,t}$, t diliminin başında açılan sunucu sayısı $y_{k,t}$ ve kapatılan sunucu sayısı $z_{k,t}$'dir. Değişkenlerin tümü negatif olmayan tamsayılardır.

3.5.2 Amaç Fonksiyonu ve Kısıtlar

Modelin amacı, çalışma, açma ve kapatma maliyetlerinin toplamını en küçüklemektir (Eşitlik 3.23).

$$min \sum_{t=1}^{H} \sum_{k \in K} \left(c_k x_{k,t} + s_k y_{k,t} + d_k z_{k,t} \right) \tag{3.23}$$

Her dilimde çalışan sunucuların işlemci ve bellek kapasitesi, tahmin edilen tepe talebi güvenlik payıyla birlikte karşılamalıdır (Eşitlik 3.24 ve 3.25).

$$\sum_{k \in K} \rho^{cpu} C_k x_{k,t} \geq (1+\alpha) \hat{D}^{cpu}_t \; \forall t \tag{3.24}$$

$$\sum_{k \in K} \rho^{mem} M_k x_{k,t} \geq (1+\alpha) \hat{D}^{mem}_t \; \forall t \tag{3.25}$$

Ardışık dilimlerdeki sunucu sayıları, açılan ve kapatılan sunucu sayılarıyla Eşitlik 3.26'daki durum geçişi kısıtı üzerinden bağlanmıştır.

$$x_{k,t} - x_{k,t-1} = y_{k,t} - z_{k,t}, x_{k,0} = x_k^0 \forall k, t \tag{3.26}$$

Minimum çalışma süresi kısıtı (Eşitlik 3.27), son L_k dilim içinde açılan sunucuların t diliminde hâlâ çalışıyor olmasını zorunlu kılar. İkinci toplam yalnızca $t < L_k$ iken boş değildir ve önceki günden devralınan, süresi henüz dolmamış açılışları temsil eder. $L_k = 1$ olduğunda kısıt, aynı dilimde açılan bir sunucunun aynı dilimde kapatılamaması koşuluna indirgenir.

$$\sum_{\tau=max(1,t-L_k+1)}^{t} y_{k,\tau} + \sum_{\tau=t-L_k+1}^{0} \hat{y}_{k,\tau} \leq x_{k,t} \forall k, t \tag{3.27}$$

$$x_{k,t}, y_{k,t}, z_{k,t} \in Z_{\geq 0} \forall k, t \tag{3.28}$$

Açma maliyeti bir veri değil, bir modelleme parametresidir. Bu parametre, yeni açılan bir sunucunun hizmete hazır olana kadar geçen ısınma süresinin maliyetini temsil edecek biçimde $s_k = c_k \cdot w$ olarak tanımlanmıştır; burada w ısınma süresinin saat cinsinden karşılığıdır. Kapatma maliyeti $d_k = 0$ alınmıştır. $s_k + d_k > 0$ olduğunda en iyi çözümde aynı (k, t) için $y_{k,t}$ ve $z_{k,t}$ birlikte pozitif olmaz. $s_k = d_k = 0$ ve $L_k = 1$ özel durumunda model, her saat için bağımsız çözülen statik bir modele indirgenir. Ana deneylerde kullanılan merkez parametre değerleri ve duyarlılık analizinde taranan değerler Tablo 3.4'te verilmiştir.

Tablo 3.4 ILP modelinin parametreleri

Parametre	Merkez değer	Duyarlılık ızgarası
Güvenlik payı α	0,05	0; 0,05; 0,10; 0,20
Açma maliyeti katsayısı w ($s_k = c_k \cdot w$)	0,25	0; 0,25; 0,5; 1
Minimum çalışma süresi L_k (dilim)	2	1; 2; 4
Kapatma maliyeti d_k	0	—
Kullanılabilir kapasite oranı ρ^{cpu}, ρ^{mem}	1	—
Planlama ufku H (saat)	24	—
Çözücü zaman sınırı (s)	60	—

3.5.3 Kayan Pencere Uygulaması

Model her gün için 24 saatlik ufukla bir kez çözülür ve elde edilen planın tamamı uygulanır. Bir sonraki günün başlangıç durumu, yani x^0_k ve $\hat{y}_{k,\tau}$ değerleri, bir önceki günün uygulanan planından devralınır. Böylece günler arasında süreklilik sağlanır ve minimum çalışma süresi kısıtı gün sınırlarında da korunur. Kayan pencere, doğrulama döneminin ilk gününden hiç sunucu çalışmıyormuş gibi ($x^0 = 0$) başlatılacak; doğrulama günleri ısınma dönemi olarak kullanılacak, başarım ölçütleri yalnızca test günlerinde hesaplanacaktır. Böylece ilk test gününün başlangıç durumu da bir önceki günün planından gelecektir. Her günün sonunda planı erken kapatma eğilimi (ufuk sonu etkisi) bilinen bir sınırlılıktır.

3.5.4 Çözüm Yöntemi ve Doğrulama

Model, Google OR-Tools kütüphanesinin doğrusal çözücü arayüzü üzerinden SCIP çözücüsüyle çözülmüştür [22, 23]. Çözüm durumu en iyi (OPTIMAL) değilse işlem hata ile durdurulmuştur. Her çözüm, modelden bağımsız yazılmış bir doğrulayıcı fonksiyonla denetlenmiş ve Eşitlik 3.24–3.28'deki kısıtların tümünün sağlandığı kontrol edilmiştir. Ayrıca küçük test örneklerinde aynı model, maliyetler tamsayıya ölçeklenerek OR-Tools'un CP-SAT çözücüsüyle de çözülmüş ve iki çözücünün aynı en iyi maliyeti verdiği birim testleriyle doğrulanmıştır.

Tamsayı kısıtlarının önemini ölçmek için her ILP çözümüne karşılık modelin doğrusal gevşetmesi de GLOP çözücüsüyle çözülecektir. Gevşetilmiş çözümdeki $x_{k,t}$ değerleri yukarı yuvarlanacak, açılan ve kapatılan sunucu sayıları yuvarlanmış plandan türetilecektir. Yukarı yuvarlama kapasite kısıtlarını korur; ancak minimum çalışma süresi kısıtının sağlanması garanti olmadığından bu kısıt ayrıca denetlenecektir. Yuvarlanmış planın maliyeti ILP maliyetiyle karşılaştırılacaktır.

3.6 Karşılaştırma Stratejileri

Tahmine dayalı ILP planı, aynı gerçek talep üzerinde beş strateji ile karşılaştırılacaktır. Stratejilerin özeti Tablo 3.5'te verilmiştir. Tepkisel stratejilerde t dilimine ait karar yalnızca t − 1 dilimine kadar gözlenen gerçek talebe dayanır. Tüm stratejiler ILP ile aynı açma maliyeti ve minimum çalışma süresi parametrelerini kullanır. Simülasyon doğrulama döneminin başından soğuk başlangıçla başlatılacak, ölçütler yalnızca test döneminde hesaplanacaktır.

Kahin ILP: Model, tahmin yerine gerçek talep D_t ile çözülür. Mükemmel öngörü altında ulaşılabilecek en düşük maliyeti verir ve regret hesabında referans olarak kullanılır.

R0 Tepkisel-ILP: Model her dilimde ufku 1 olacak biçimde, girdi olarak bir önceki dilimin gerçekleşen talebi D_{t-1} ile çözülür. Tahminin sağladığı değeri, aynı optimizasyon modelini kullanarak yalıtır.

R1 Eşik kuralı: Kubernetes HPA'sına benzer bir kuraldır. Yalnızca tek bir ölçek tipi (m8i.xlarge) kullanılır ve en az bir sunucu çalıştırılır.

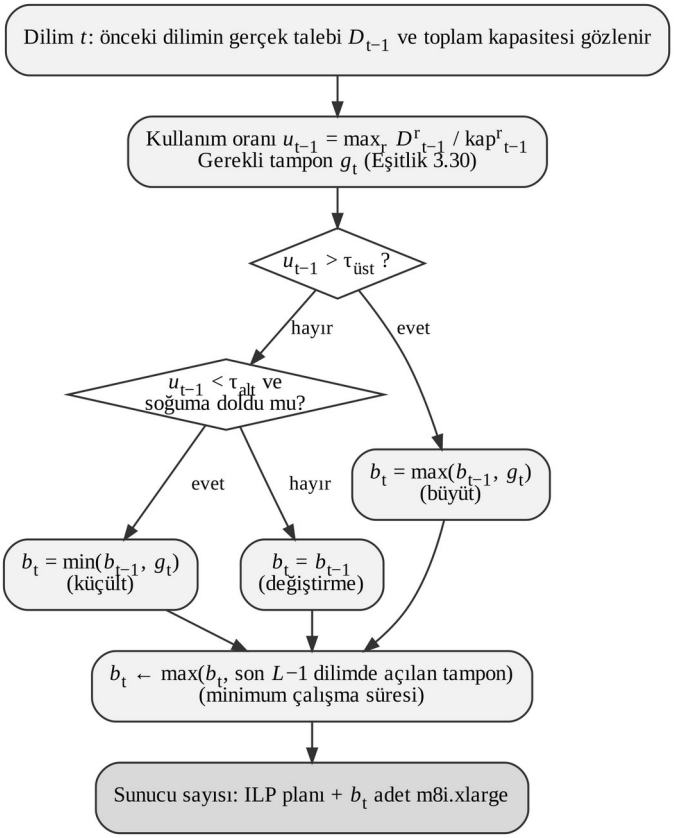
R2 Hibrit: Tahmine dayalı ILP planının üzerine, R1 ile aynı kuralla yönetilen ve m8i.xlarge tipinden oluşan bir tampon eklenir. İki varyantı vardır: R1 ile aynı HPA eşiklerini kullanan R2 (HPA eşikleri) ve ILP planının hedef kullanımına uyumlu eşikleri kullanan R2 (uyumlu eşikler).

Eşik kuralında bir önceki dilimin kullanım oranı, işlemci ve bellek kullanım oranlarının büyüğü olarak Eşitlik 3.29'daki gibi hesaplanmıştır. Burada kap^{r}_{t-1}, $t-1$ dilimindeki toplam kapasitedir. Gerekli tampon büyüklüğü g_t, ILP planının t dilimindeki kapasitesiyle birlikte D_{t-1} talebini hedef kullanım oranında karşılayacak en küçük ölçek tipi sunucu sayısıdır (Eşitlik 3.30). Eşitlikte q^r ölçek tipinin r kaynağındaki kapasitesi, $kap^{r,ILP}_{t}$ ise ILP planının kapasitesidir; R1'de ILP planı yoktur ve bu terim sıfırdır.

$$u_{t-1} = \max_{r \in \{cpu,mem\}} \frac{D^r_{t-1}}{k a p^r_{t-1}} \tag{3.29}$$

$$g_t = max\left(0, \lceil \max_r \frac{D^r_{t-1}/\tau_{hedef} - k a p^{r,ILP}_t}{q^r} \rceil\right) \tag{3.30}$$

Tamponun güncellenmesi Şekil 3.2'de görüldüğü gibidir. Kullanım üst eşiği aşarsa tampon gerekli büyüklüğe çıkarılır. Kullanım alt eşiğin altına düşerse ve son ölçek değişikliğinden bu yana soğuma süresi dolmuşsa tampon gerekli büyüklüğe indirilir. Diğer durumlarda tampon değiştirilmez. Son $L-1$ dilimde açılan tampon sunucuları minimum çalışma süresi nedeniyle kapatılamaz. R2'de tampon sıfırın altına inmez; dolayısıyla sunucu sayısı hiçbir zaman ILP planının altına düşmez.



Şekil 3.2 R2 hibrit stratejisinde tamponun güncellenmesi

R2'nin iki varyantının gerekçesi şudur: ILP planı kapasiteyi tahminin (1 + α) katına göre ayarladığından, tahmin doğru olduğunda kullanım oranı yaklaşık 1/(1 + α) ≈ 0,95 düzeyinde olur. HPA'nın varsayılan üst eşiği olan 0,8 bu durumda neredeyse her dilimde aşılır. Tampon da 0,5 olan alt eşiğin altına inemediği için küçülmez. Bu nedenle ikinci varyantta yalnızca gerçek talep kapasiteyi aştığında büyüyen ($\tau_{üst}$ = 1) ve hedefi ILP'nin güvenlik payıyla aynı olan (τ_{hedef} = 1/(1 + α)) eşikler tanımlanmıştır. Stratejilerin eşik değerleri Tablo 3.5'te verilmiştir.

Tablo 3.5 Karşılaştırma stratejileri ve eşik parametreleri

Strateji	**Girdi**	**$\tau_{üst}$**	**τ_{hedef}**	**τ_{alt}**	**Soğuma (dilim)**
Tahmine dayalı ILP	$\hat{D}_t$ (tahmin)	—	—	—	—
Kahin ILP	D_t (gerçek)	—	—	—	—
R0 Tepkisel-ILP	D_{t-1}	—	—	—	—
R1 Eşik kuralı	D_{t-1}	0,8	0,7	0,5	2
R2 Hibrit (HPA eşikleri)	$\hat{D}_t + D_{t-1}$	0,8	0,7	0,5	2
R2 Hibrit (uyumlu	$\hat{D}_t + D_{t-1}$	1,0	1/(1+α)	0,8	2

Strateji	Girdi	$\tau_{üst}$	τ_{hedef}	τ_{alt}	Soğuma (dilim)
eşikler)					

3.7 Değerlendirme Ölçütleri

Tüm stratejiler test dönemindeki gerçek talep üzerinde, güvenlik payı dahil edilmeden hesaplanan kapasiteyle değerlendirilecektir. r ∈ {cpu, mem} kaynağı için t dilimindeki kapasite Eşitlik 3.31 ile hesaplanır (bellek için C_k yerine M_k kullanılır).

$$kap_t^{cpu} = \sum_{k \in K} \rho^{cpu} C_k x_{k,t} \tag{3.31}$$

Kullanılan ölçütler şunlardır:

(i) Toplam maliyet: Eşitlik 3.23'teki çalışma, açma ve kapatma maliyetlerinin test dönemindeki toplamıdır. Tablolarda günlük ortalama maliyet olarak verilecektir.

(ii) Regret: Stratejinin maliyetinin aynı havuzdaki kahin ILP maliyetine göre yüzde farkıdır (Eşitlik 3.32). Negatif regret, stratejinin kahinden daha ucuz olduğunu gösterir. Bu yalnızca kahinin karşıladığı talebin bir kısmının karşılanmamasıyla mümkündür; bu nedenle regret her zaman eksik kapasite ölçütleriyle birlikte yorumlanmalıdır.

(iii) Eksik kapasiteli saat oranı: İşlemci veya bellek kapasitesinden en az birinin gerçek talebin altında kaldığı dilimlerin oranıdır (Eşitlik 3.33).

(iv) Karşılanmayan talep: Kapasitenin karşılayamadığı talebin toplam talebe oranıdır (Eşitlik 3.34) ve işlemci ile bellek için ayrı hesaplanır.

(v) Boşta kapasite oranı: Kapasitenin kullanılmayan kısmının ortalama oranıdır (Eşitlik 3.35). Kapasitenin yetersiz kaldığı dilimlerde boşta kapasite sıfır kabul edilir.

(vi) Olay süresi: Art arda eksik kapasiteli dilimler bir olay olarak sayılır ve olayların en uzun süresi raporlanır. Bu ölçüt, bir stratejinin kapasite açığını kaç saatte kapattığını gösterir.

(vii) Açılış sayısı: Test döneminde açılan toplam sunucu sayısıdır.

$$Regret = 100 \frac{Maliyet - Maliyet^{kahin}}{Maliyet^{kahin}} \tag{3.32}$$

$$ESO = \frac{1}{T}\sum_{t=1}^{T} 1\left[D_t^{cpu} > kap_t^{cpu} \vee D_t^{mem} > kap_t^{mem}\right] \tag{3.33}$$

$$KT^r = 100\frac{\sum_{t=1}^{T} max\left(D_t^r - kap_t^r, 0\right)}{\sum_{t=1}^{T} D_t^r} \tag{3.34}$$

$$BKO^r = \frac{1}{T}\sum_{t=1}^{T} \frac{max\left(kap_t^r - D_t^r, 0\right)}{kap_t^r} \tag{3.35}$$

Kahin ILP de (1 + α) güvenlik payıyla planlandığından, α arttıkça kahin maliyeti de artar. Bu nedenle duyarlılık analizinde regret, aynı (w, L) değerleri için α = 0 ile çözülen kahin maliyetine göre de hesaplanacaktır (alt sınıra göre regret).

3.8 Duyarlılık Analizi ve Sentetik Senaryo Tasarımı

Ana deneyler Tablo 3.4'teki merkez değerlerle yapılacaktır. Sonuçların bu değerlere ne kadar bağlı olduğunu görmek için α, w ve L parametrelerinin Tablo 3.4'teki tüm birleşimlerinden oluşan 48 noktalık tam faktöriyel bir ızgara taranacaktır. Her ızgara noktasında tahmine dayalı ILP (tüm tahmin modelleriyle), kahin ILP, R0, R1 ve R2'nin iki varyantı çalıştırılacaktır. R2 (uyumlu eşikler) varyantının hedef kullanımı her noktada o noktanın α değerine göre belirlenecek, regret her noktanın kendi kahinine göre hesaplanacaktır. Alibaba bellek varsayımına duyarlılık ise tüm deneylerin $r \in \{2, 4, 8\}$ için tekrarlanmasıyla incelenecektir (Bölüm 3.3.3).

Gerçek veride bulunmayan ani talep değişimlerinin etkisini incelemek için kontrollü sentetik senaryolar oluşturulacaktır. Bozulmalar yalnızca test dönemine, 5 dakikalık havuz toplamı serisine uygulanacak ve saatlik tepe değerleri bozulmuş seriden yeniden hesaplanacaktır. Tahmin modelleri bozulmamış eğitim ve doğrulama verisiyle, ana deneylerde seçilen SARIMA dereceleri ve LSTM epoch sayılarıyla yeniden eğitilecek, ardından bozulmuş seride parametreleri sabit tutularak ileri yürüyen tahmin yapılacaktır. Daha sonra ILP, R0, R1 ve R2 merkez parametrelerle çalıştırılacaktır. Planlanan senaryolar Tablo 3.6'da verilmiştir. Bozulmanın başlangıç saati her senaryo ve veri kümesi için sabit bir tohumla (2026) seçilecek ve aynı veri kümesinin tüm havuzlarında ortak tutulacaktır. Sentetik senaryo sonuçları gerçek veri sonuçlarıyla karıştırılmadan ayrı tablolarda raporlanacaktır.

Tablo 3.6 Sentetik senaryolar

Senaryo	Uygulanan bozulma
Kontrol	Bozulma yok; ana deneylerle tutarlılığı ölçer
Basamak	Başlangıçtan test sonuna kadar talep × 1,5 (etki en az 12 saat sürer)
Darbe	3 saat boyunca talep × 2
Ani düşüş	Başlangıçtan test sonuna kadar talep × 0,5 (etki en az 12 saat sürer)
Mevsimsellik bozulması	Test dilimlerinde her değer 12 saat önceki değerle değiştirilir

4. İLK BEŞ HAFTADA TAMAMLANAN ÇALIŞMALAR VE ÖN SONUÇLAR

Bu bölümde çalışmanın ilk beş haftasında tamamlanan işler özetlenmiş, ardından veri kümelerinin keşifsel analizine ve ILP modelinin ilk uygulamasına ait ön sonuçlar verilmiştir. Bölümün sonunda bu aşamada karşılaşılan sorunlar ve bunlar için benimsenen çözüm yaklaşımları açıklanmıştır. Çalışma bireysel olarak yürütülmüş, tüm aşamalar rapor yazarı tarafından gerçekleştirilmiştir. Tablolarda ondalık ayırıcı olarak virgül kullanılmıştır.

4.1 Tamamlanan Çalışmalar

İlk beş haftada tamamlanan çalışmalar şunlardır:

(i) Konu ve kapsam: Çalışmanın konusu ve kapsamı danışman ile birlikte belirlenmiş, araştırma soruları tanımlanmıştır (Bölüm 1.3).

(ii) Literatür taraması: Bulut iş yükü tahmini, kapasite planlamasında tamsayılı programlama ve otomatik ölçekleme alanlarındaki çalışmalar incelenmiş ve bu çalışmanın bileşenleriyle karşılaştırılmıştır (Bölüm 2).

(iii) Veri edinimi ve ön işleme: Bitbrains GWA-T-12 ve Alibaba cluster-trace-v2018 izleri indirilmiş, veri kalitesi incelenmiştir. Zaman hizalama, birim dönüşümleri, eksik veri kuralları, havuz oluşturma ve saatlik tepe talep hesabı yazılım olarak gerçekleştirilmiş; 7 Bitbrains ve 24 Alibaba havuz serisi elde edilmiştir (Bölüm 3.2–3.3).

(iv) Keşifsel veri analizi: Tepe ve ortalama talep, talebin işlemci–bellek bileşimi, mevsimsellik ve durağanlık incelenmiştir (Bölüm 4.2).

(v) Tahmin modelleri: Mevsimsel naif ve son değer taban modelleri, SARIMA derece seçim yordamı ve LSTM modeli kodlanmıştır. SARIMA dereceleri ve LSTM eğitim ayarları yalnızca eğitim ve doğrulama verisi kullanılarak belirlenmiştir (Bölüm 3.4).

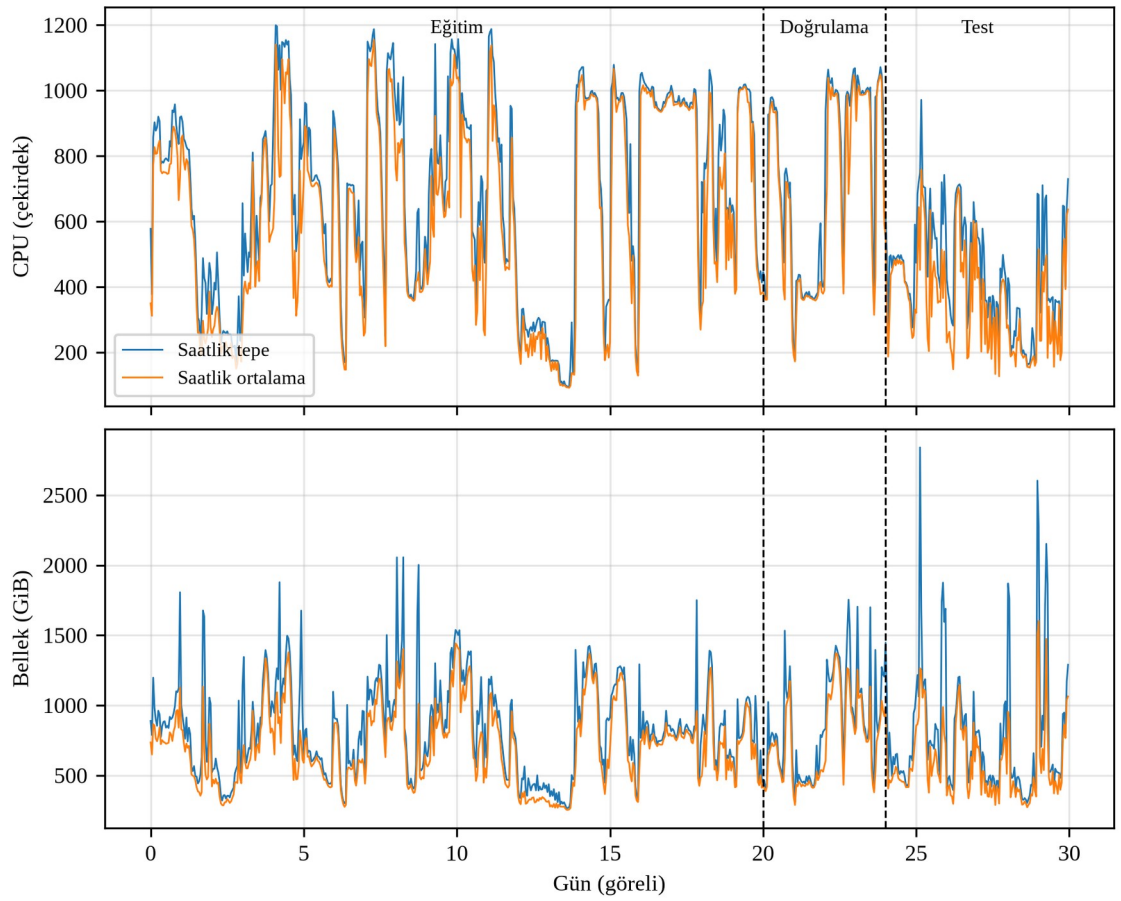
(vi) Sunucu tipleri ve fiyatlar: Planlamada kullanılacak üç AWS EC2 sunucu tipi seçilmiş, fiyatları AWS fiyat listesi programlama arayüzünden alınarak sabitlenmiştir (Tablo 3.3).

(vii) ILP modeli: Çok dönemli kapasite planlama modeli formüle edilmiş (Eşitlik 3.23–3.28) ve Google OR-Tools ile kodlanmıştır. Çözümleri denetleyen modelden bağımsız bir doğrulayıcı ile küçük örneklerde SCIP ve CP-SAT çözümlerini karşılaştıran birim testleri yazılmıştır (Bölüm 3.5, Ek A).

(viii) Karşılaştırma ve değerlendirme tasarımı: Kahin ILP, tepkisel ve hibrit stratejiler, değerlendirme ölçütleri, duyarlılık analizi ve sentetik senaryolar tanımlanmıştır (Bölüm 3.6–3.8).

4.2 Keşifsel Veri Analizi

Bitbrains fastStorage havuzunun saatlik tepe ve saatlik ortalama talebi Şekil 4.1'de görüldüğü gibidir. Talep, gün içi döngülerden çok birkaç gün süren düzey değişimleriyle karakterize edilir. İşlemci talebi eğitim döneminde yaklaşık 100 ile 1.200 çekirdek arasında değişmiş, test döneminde ise daha düşük bir düzeye inmiştir. Bellek talebinde kısa süreli ve yüksek sıçramalar dikkat çekmektedir. Test döneminde bu sıçramaların genliği eğitim dönemindekinden büyüktür; örneğin bellek tepe talebi test döneminde 2.500 GiB'ı aşmıştır. Test döneminin eğitim döneminden farklı bir davranış göstermesi, tahmin modellerini zorlayacak önemli bir etkendir.



Şekil 4.1 Bitbrains fastStorage havuzunun saatlik tepe ve saatlik ortalama işlemci ve bellek talebi

Tepe talebin ortalama talebe oranı Tablo 4.1'de verilmiştir. Oran, her dönemde saatlik tepe değerlerin toplamının saatlik ortalama değerlerin toplamına bölünmesiyle hesaplanmıştır. Tüm havuz ve dönemlerde oran 1'den büyüktür; saat içi tepe, saatlik ortalamayı Bitbrains'te %7 ile %37, Alibaba'da işlemci için %17 ve bellek için %2 oranında aşmıştır. Oran, fastStorage havuzunun test döneminde en yüksek değerine ulaşmıştır (işlemci 1,261, bellek 1,314). Saatlik oranların dağılımı daha geniştir: fastStorage test döneminde saatlik oranın %95'lik yüzdeliği bellek için 1,96'dır, yani bazı

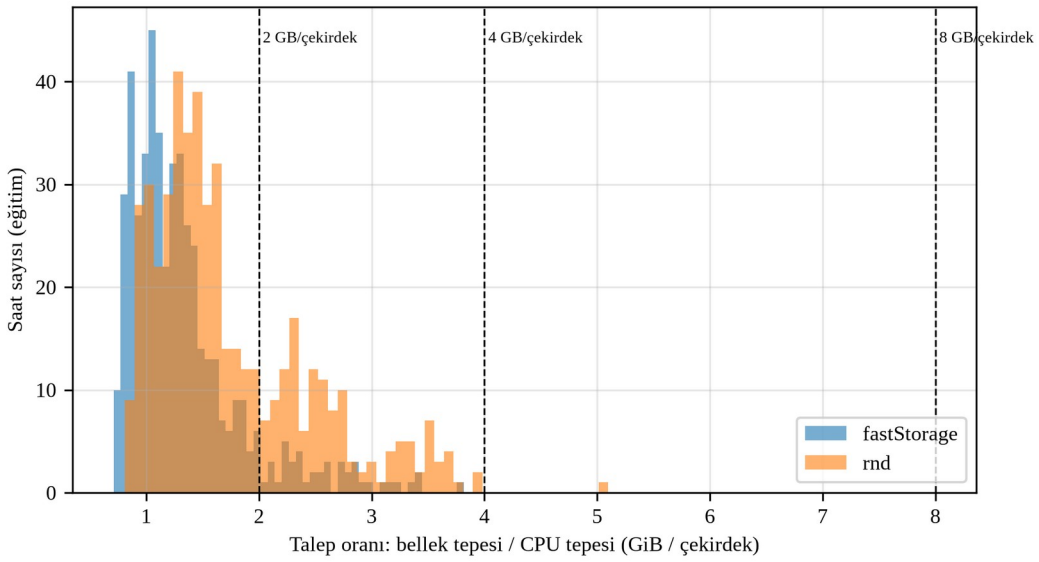
saatlerde tepe talep ortalamanın yaklaşık iki katına çıkmıştır. Bu bulgu, planlamanın ortalama yerine tepe talebe göre yapılmasının gerekli olduğunu göstermektedir.

Tablo 4.1 Toplam saatlik tepe talebin toplam saatlik ortalama talebe oranı

Grup	Eğitim işlemci	Eğitim bellek	Doğrulama işlemci	Doğrulama bellek	Test işlemci	Test bellek
Bitbrains fastStorage	1,125	1,179	1,083	1,169	1,261	1,314
Bitbrains Rnd	1,192	1,194	1,129	1,163	1,072	1,127
fastStorage alt havuzları	1,133 ± 0,016	1,205 ± 0,013	1,088 ± 0,010	1,201 ± 0,018	1,276 ± 0,027	1,344 ± 0,022
Alibaba (8 havuz)	1,171	1,022	—	—	—	—

Not: Alibaba için oran yalnızca eğitim döneminde hesaplanmıştır; bellek oranı r değerinden bağımsızdır.

Talebin işlemci ve bellek bileşenleri arasındaki oran Şekil 4.2'de, bu orana göre hangi kaynağın bağlayıcı olduğu ise Tablo 4.2'de verilmiştir. Bitbrains havuzlarında talep, çekirdek başına ortalama 1,2–1,5 GiB bellek gerektirmektedir. Bu oran, en düşük bellek oranlı hesaplama optimize ailenin (2 GB/vCPU) bile altındadır; dolayısıyla Bitbrains'te kapasiteyi çoğunlukla işlemci talebi belirler. Hesaplama optimize ailede bellek, fastStorage saatlerinin yalnızca %9,2'sinde, Rnd saatlerinin ise %28,5'inde bağlayıcı olmuştur. Bu nedenle Bitbrains'te ILP'nin ağırlıklı olarak c8i.xlarge tipini seçmesi beklenir; Bölüm 4.3'teki ilk uygulama bu beklentiyi doğrulamıştır.



Şekil 4.2 Bitbrains havuzlarında bellek tepe talebinin işlemci tepe talebine oranının dağılımı (eğitim dönemi)

Alibaba'da ise durum tersidir. Makineler bellek yoğun çalıştığından çekirdek başına bellek talebi r = 2 varsayımında bile 4,38 GB'tır ve bellek, hesaplama optimize ailede saatlerin tamamında bağlayıcıdır. r = 8 varsayımında talep çekirdek başına 17,5 GB'a çıkar ve en yüksek bellek oranlı aileyi (8 GB/vCPU) bile aşar. Bu durumda bellek kısıtını karşılamak için açılan sunucuların işlemci kapasitesinin büyük kısmı boşta kalacaktır. Bu

sonuç, Alibaba bulgularının M_{ref} varsayımına güçlü biçimde bağlı olacağını ve bu nedenle duyarlılık analizinin gerekli olduğunu göstermektedir.

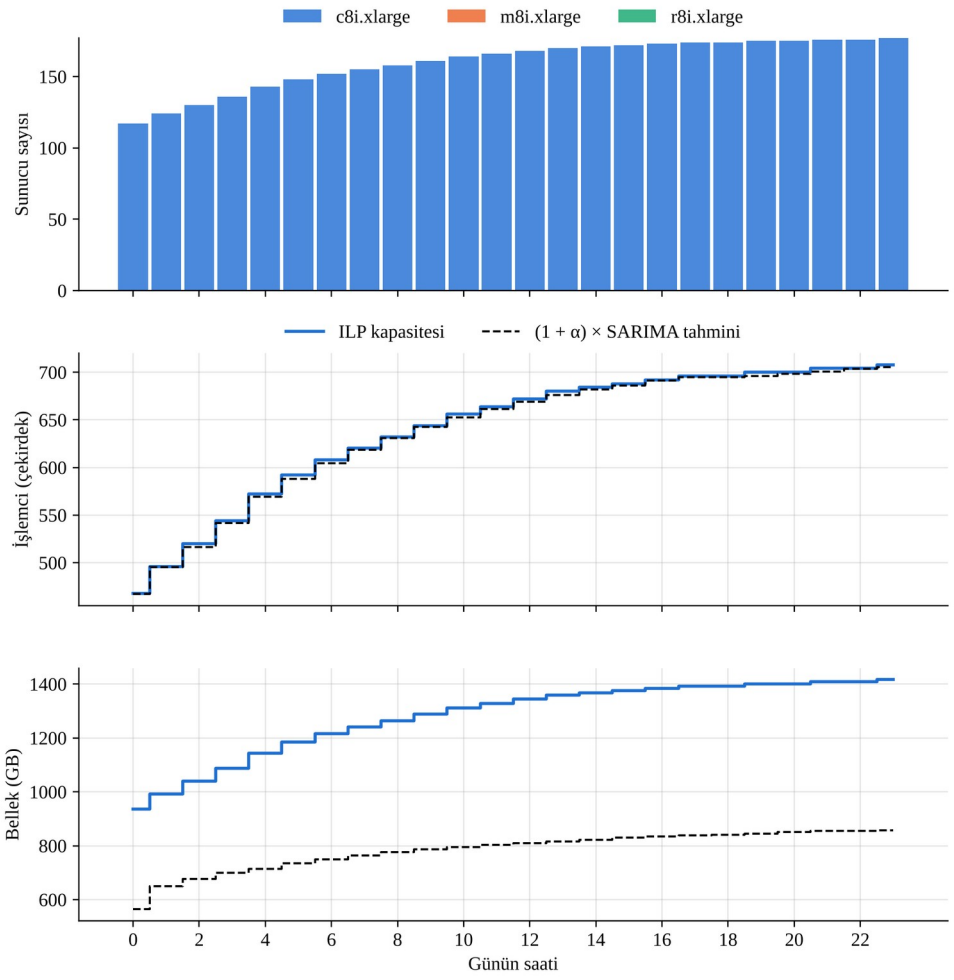
Tablo 4.2 Talebin bellek/işlemci oranı (GB/çekirdek) ve bellek kısıtının bağlayıcı olduğu saatlerin oranı (eğitim dönemi)

Grup	Oran medyan	Oran %5	Oran %95	Bellek bağlayıcı, 2 GB/vCPU (%)	Bellek bağlayıcı, 4 GB/vCPU (%)	Bellek bağlayıcı, 8 GB/vCPU (%)
Bitbrains fastStorage	1,20	0,80	2,43	9,2	0,0	0,0
Bitbrains Rnd	1,51	0,93	3,30	28,5	0,2	0,0
fastStorage alt havuzları	1,22 ± 0,08	0,80 ± 0,04	2,48 ± 0,28	10,7 ± 2,8	0,2 ± 0,1	0,0 ± 0,0
Alibaba (r = 2)	4,38	3,03	6,20	100,0	61,5	0,0
Alibaba (r = 4)	8,75	6,06	12,39	100,0	100,0	61,5
Alibaba (r = 8)	17,50	12,11	24,79	100,0	100,0	100,0

Mevsimsellik ve durağanlık analizleri, iki veri kümesinin zaman yapısının da farklı olduğunu göstermiştir. Bitbrains havuzlarında STL mevsimsellik gücü işlemci için 0,20–0,25, bellek için 0,16–0,22 aralığında kalmıştır. Toplam serinin bir gün gecikmeli otokorelasyonu da fastStorage işlemci talebi için yalnızca 0,12'dir. Dolayısıyla Bitbrains'te günlük döngü zayıftır. Alibaba işlemci talebinde ise mevsimsellik gücü yaklaşık 0,83'tür ve belirgin bir günlük döngü vardır; Alibaba bellek talebinde bu değer 0,27–0,47 aralığındadır. Durağanlık testleri fastStorage serileri ve Alibaba işlemci serileri için d = 0, Rnd ve Alibaba bellek serileri için d = 1 sonucunu vermiştir. Bu sonuçlar SARIMA derece seçiminde kullanılmıştır (Bölüm 3.4.3).

4.3 ILP Modelinin İlk Uygulaması

Kurulan ILP modelinin doğru çalıştığını görmek amacıyla model, Bitbrains fastStorage havuzunda doğrulama döneminin ilk günü için SARIMA modelinin gün-ileri tahminleriyle çözülmüştür. Çözümde Tablo 3.4'teki merkez parametreler ($\alpha = 0{,}05$, $w = 0{,}25$, $L = 2$) kullanılmış ve gün boş bir filoyla ($x^0 = 0$) başlatılmıştır. Elde edilen plan Şekil 4.3'te görüldüğü gibidir.



Şekil 4.3 Bitbrains fastStorage havuzunda bir günlük ILP planı: sunucu sayıları ile kapasite ve güvenlik paylı SARIMA tahmini

SCIP çözücüsü problemi yaklaşık 0,06 saniyede en iyi çözüme ulaştırmış ve plan, modelden bağımsız doğrulayıcının tüm denetimlerinden geçmiştir. Planın açma maliyeti dahil günlük maliyeti 829,7 USD'dir. Plan yalnızca hesaplama optimize c8i.xlarge sunuculardan oluşmaktadır. İşlemci kapasitesi her saatte güvenlik paylı tahminin hemen üzerindedir; yani işlemci kısıtı bağlayıcıdır. Bellek kapasitesi ise tahminin belirgin biçimde üzerinde kalmıştır. Bu sonuç Bölüm 4.2'deki bulguyla uyumludur: talep çekirdek başına 2 GB'tan az bellek gerektirdiğinden en ucuz çözüm, vCPU başına en ucuz tip olan c8i.xlarge ile elde edilmektedir. Sunucu sayısının gün içinde 117'den 177'ye kademeli olarak artması, tahmin edilen işlemci talebindeki artışı izlemektedir.

Bu ilk uygulama, modelin kısıtları doğru biçimde temsil ettiğini ve günlük planlamada hesaplama süresinin bir sorun oluşturmayacağını göstermektedir. Planların gerçekleşen talep karşısındaki başarımının ölçülmesi ve diğer stratejilerle karşılaştırılması sonraki aşamada yapılacaktır (Bölüm 5).

4.4 Karşılaşılan Sorunlar ve Çözüm Yaklaşımları

İlk beş haftada karşılaşılan başlıca sorunlar ve bunlar için benimsenen çözümler şunlardır:

(i) Alibaba izinin başındaki ölçüm boşlukları: İzin ilk yaklaşık 39 saatinde makinelerin önemli bir kısmı ölçüm göndermemiş, ardından küme genelinde bir ölçüm kesintisi yaşanmıştır. İlk iki gün analizden çıkarılarak uzun boşluk oranı %8,76'dan %0,15'e indirilmiştir (Bölüm 3.2.2).

(ii) Alibaba'da fiziksel bellek miktarının bilinmemesi: Bellek kullanımı yalnızca yüzde olarak verildiğinden, çekirdek başına bellek katsayısına dayanan bir referans bellek varsayımı tanımlanmış ve üç farklı değerle duyarlılık analizi yapılması kararlaştırılmıştır (Bölüm 3.3.3).

(iii) Zaman damgası biriminin belirsizliği: Zaman damgasının birimi sütun adına güvenilmeden, ardışık ölçümler arasındaki farkların medyanından belirlenmiştir (Bölüm 3.3.1).

(iv) Tepe talebin hesaplanma sırası: Makine bazında tepelerin toplanmasının talebi olduğundan yüksek gösterdiği belirlenmiş, tepe değer havuz toplamı üzerinden hesaplanmıştır (Bölüm 3.3.4).

(v) Alibaba'da LSTM için eğitim örneği azlığı: 96 saatlik eğitim dönemi 48 saatlik bakış penceresiyle yalnızca 25 eğitim örneği verdiğinden, bakış penceresi 24 saate indirilerek örnek sayısı 49'a çıkarılmıştır (Bölüm 3.4.4).

(vi) HPA eşiklerinin güvenlik paylı planla uyumsuzluğu: Varsayılan HPA eşiklerinin ILP planı üzerinde tamponu sürekli devrede tutacağı görülmüş, plana uyumlu eşikler tanımlanmıştır (Bölüm 3.6).

5. SONRAKİ AŞAMALAR İÇİN ÇALIŞMA PLANI

Bu bölümde çalışmanın 6.–15. haftalarında yapılması planlanan işler verilmiştir. Plan, Bölüm 1.3'teki araştırma sorularını yanıtlamaya yönelik deneyleri, bu deneylerin sonuçlarına göre ele alınacak geliştirmeleri ve raporlama aşamalarını kapsamaktadır. Haftalık plan Tablo 5.1'de özetlenmiştir.

Tablo 5.1 6.–15. haftalar için çalışma planı

Hafta	Planlanan çalışma	İlgili soru / çıktı
6	Tahmin modellerinin test döneminde değerlendirilmesi (MAE, RMSE, MAPE, MASE, ETO) ve modellerin karşılaştırılması	Soru (i)
7	ILP planlarının tüm havuzlarda kayan pencereyle üretilmesi; kahin ILP; LP gevşetmesi ve yuvarlama ile karşılaştırma	Soru (ii), (iii)
8	Tepkisel (R0, R1) ve hibrit (R2) stratejilerin benzetimi ve tahmine dayalı ILP ile karşılaştırılması	Soru (iv)
9	Duyarlılık analizi (α, w, L ve Alibaba için r) ve sentetik senaryo deneyleri	Soru (ii), (iv)
10	Belirsizlik altında planlama yaklaşımlarının incelenmesi: kayan ufuklu yeniden planlama, kantil tahmine dayalı güvenlik payı ve iki aşamalı stokastik programlama	Geliştirme
11	İkinci ara raporun hazırlanması ve teslimi	2. ara rapor
12	Sonuçların istatistiksel anlamlılığının sınanması (Diebold–Mariano testi, bootstrap güven aralıkları) ve sunucu açılış gecikmesine duyarlılık	Sağlamlık
13	Planın bir Kubernetes kümesinde uygulanmasını gösteren prototip (benzetimli düğümlerle)	Uygulama
14	Ek geliştirmeler (rezerve ve spot fiyatlandırma, ek veri kümesi) ve sonuçların bütünleştirilmesi	Geliştirme
15	Son raporun tamamlanması ve teslimi	Son rapor

6.–9. haftalardaki deneyler, Bölüm 3'te tanımlanan yöntem ve ölçütlerle doğrudan araştırma sorularına yanıt verecektir. 10. haftada ele alınacak yaklaşımlar, sabit güvenlik payı α yerine talep belirsizliğini planlamaya daha doğrudan katmayı amaçlamaktadır. 12.–14. haftalardaki çalışmalar ise sonuçların sağlamlığını ve uygulanabilirliğini sınamaya yöneliktir; bu çalışmaların kapsamı temel deneylerin sonuçlarına göre önceliklendirilecektir.

İkinci ara rapor, bu raporun danışman geri bildirimleri doğrultusunda geliştirilmiş hâli olarak hazırlanacak ve ilk on haftada elde edilen sonuçları, karşılaşılan sorunları ve dönem sonuna kadar tamamlanacak işleri içerecektir.

KAYNAKLAR

[1] Armbrust, M. ve Fox, A., et al., (2010), "A View of Cloud Computing", Communications of the ACM, Vol.53, No.4, pp.50-58.

[2] Burns, B. ve Grant, B., et al., (2016), "Borg, Omega, and Kubernetes", Communications of the ACM, Vol.59, No.5, pp.50-57.

[3] Reiss, C. ve Tumanov, A., et al., (2012), "Heterogeneity and Dynamicity of Clouds at Scale: Google Trace Analysis", Third ACM Symposium on Cloud Computing (SoCC '12), San Jose, pp.1-13.

[4] The Kubernetes Authors, (2026), "Horizontal Pod Autoscaling", https://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/, (Erişim tarihi: 02.11.2026).

[5] Lorido-Botran, T. ve Miguel-Alonso, J., et al., (2014), "A Review of Auto-scaling Techniques for Elastic Applications in Cloud Environments", Journal of Grid Computing, Vol.12, No.4, pp.559-592.

[6] Roy, N. ve Dubey, A., et al., (2011), "Efficient Autoscaling in the Cloud Using Predictive Models for Workload Forecasting", IEEE 4th International Conference on Cloud Computing (CLOUD), Washington, pp.500-507.

[7] Qu, C. ve Calheiros, R.N., et al., (2018), "Auto-Scaling Web Applications in Clouds: A Taxonomy and Survey", ACM Computing Surveys, Vol.51, No.4, pp.1-33.

[8] The Kubernetes Authors, (2026), "Resource Management for Pods and Containers", https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/, (Erişim tarihi: 02.11.2026).

[9] Rzadca, K. ve Findeisen, P., et al., (2020), "Autopilot: Workload Autoscaling at Google", Fifteenth European Conference on Computer Systems (EuroSys '20), Heraklion, pp.1-16.

[10] The Kubernetes Authors, (2026), "Node Autoscaling", https://kubernetes.io/docs/concepts/cluster-administration/node-autoscaling/, (Erişim tarihi: 02.11.2026).

[11] Amazon Web Services, (2026), "AWS Price List Bulk API: Amazon EC2, eu-central-1", https://pricing.us-east-1.amazonaws.com/offers/v1.0/aws/AmazonEC2/current/eu-central-1/index.csv, (Erişim tarihi: 29.10.2026).

[12] Shen, S. ve van Beek, V., et al., (2015), "Statistical Characterization of Business-Critical Workloads Hosted in Cloud Datacenters", 15th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (CCGrid), Shenzhen, pp.465-474.

[13] Guo, J. ve Chang, Z., et al., (2019), "Who Limits the Resource Efficiency of My Datacenter: An Analysis of Alibaba Datacenter Traces", International Symposium on Quality of Service (IWQoS '19), Phoenix, pp.1-10.

[14] Box, G.E.P. ve Jenkins, G.M., et al., (2015), Time Series Analysis: Forecasting and Control, 5th ed., Wiley, Hoboken.

[15] Calheiros, R.N. ve Masoumi, E., et al., (2015), "Workload Prediction Using ARIMA Model and Its Impact on Cloud Applications' QoS", IEEE Transactions on Cloud Computing, Vol.3, No.4, pp.449-458.

[16] Hochreiter, S. ve Schmidhuber, J., (1997), "Long Short-Term Memory", Neural Computation, Vol.9, No.8, pp.1735-1780.

[17] Kumar, J. ve Goomer, R., et al., (2018), "Long Short Term Memory Recurrent Neural Network (LSTM-RNN) Based Workload Forecasting Model for Cloud Datacenters", Procedia Computer Science, Vol.125, pp.676-682.

[18] Zeng, A. ve Chen, M., et al., (2023), "Are Transformers Effective for Time Series Forecasting?", Proceedings of the AAAI Conference on Artificial Intelligence, Vol.37, No.9, pp.11121-11128.

[19] Wolsey, L.A., (2020), Integer Programming, 2nd ed., Wiley, Hoboken.

[20] Carrión, M. ve Arroyo, J.M., (2006), "A Computationally Efficient Mixed-Integer Linear Formulation for the Thermal Unit Commitment Problem", IEEE Transactions on Power Systems, Vol.21, No.3, pp.1371-1378.

[21] Lin, M. ve Wierman, A., et al., (2013), "Dynamic Right-Sizing for Power-Proportional Data Centers", IEEE/ACM Transactions on Networking, Vol.21, No.5, pp.1378-1391.

[22] Perron, L. ve Furnon, V., (2026), "OR-Tools", Sürüm 9.15, Google, https://developers.google.com/optimization/, (Erişim tarihi: 02.11.2026).

[23] Achterberg, T., (2009), "SCIP: Solving Constraint Integer Programs", Mathematical Programming Computation, Vol.1, No.1, pp.1-41.

[24] Urgaonkar, B. ve Shenoy, P., et al., (2008), "Agile Dynamic Provisioning of Multi-Tier Internet Applications", ACM Transactions on Autonomous and Adaptive Systems, Vol.3, No.1, pp.1-39.

[25] Hyndman, R.J. ve Athanasopoulos, G., (2021), Forecasting: Principles and Practice, 3rd ed., OTexts, Melbourne.

[26] Gong, Z. ve Gu, X., et al., (2010), "PRESS: PRedictive Elastic ReSource Scaling for Cloud Systems", International Conference on Network and Service Management (CNSM), Niagara Falls, pp.9-16.

[27] Imdoukh, M. ve Ahmad, I., et al., (2020), "Machine Learning-Based Auto-Scaling for Containerized Applications", Neural Computing and Applications, Vol.32, No.13, pp.9745-9760.

[28] Toka, L. ve Dobreff, G., et al., (2021), "Machine Learning-Based Scaling Management for Kubernetes Edge Clusters", IEEE Transactions on Network and Service Management, Vol.18, No.1, pp.958-972.

[29] Patil, S. ve D G, N., et al., (2025), "Dynamic Autoscaling and Scheduling in Kubernetes Clusters with LSTM and ILP", Journal of Communications Software and Systems, Vol.21, No.4, pp.465-476.

[30] Speitkamp, B. ve Bichler, M., (2010), "A Mathematical Programming Approach for Server Consolidation Problems in Virtualized Data Centers", IEEE Transactions on Services Computing, Vol.3, No.4, pp.266-278.

[31] Beloglazov, A. ve Buyya, R., (2012), "Optimal Online Deterministic Algorithms and Adaptive Heuristics for Energy and Performance Efficient Dynamic Consolidation of Virtual Machines in Cloud Data Centers", Concurrency and Computation: Practice and Experience, Vol.24, No.13, pp.1397-1420.

[32] Gandhi, A. ve Harchol-Balter, M., et al., (2012), "AutoScale: Dynamic, Robust Capacity Management for Multi-Tier Data Centers", ACM Transactions on Computer Systems, Vol.30, No.4, pp.1-26.

[33] Dickey, D.A. ve Fuller, W.A., (1979), "Distribution of the Estimators for Autoregressive Time Series with a Unit Root", Journal of the American Statistical Association, Vol.74, No.366, pp.427-431.

[34] Kwiatkowski, D. ve Phillips, P.C.B., et al., (1992), "Testing the Null Hypothesis of Stationarity Against the Alternative of a Unit Root", Journal of Econometrics, Vol.54, No.1-3, pp.159-178.

[35] Cleveland, R.B. ve Cleveland, W.S., et al., (1990), "STL: A Seasonal-Trend Decomposition Procedure Based on Loess", Journal of Official Statistics, Vol.6, No.1, pp.3-73.

[36] Wang, X. ve Smith, K., et al., (2006), "Characteristic-Based Clustering for Time Series Data", Data Mining and Knowledge Discovery, Vol.13, No.3, pp.335-364.

[37] Akaike, H., (1974), "A New Look at the Statistical Model Identification", IEEE Transactions on Automatic Control, Vol.19, No.6, pp.716-723.

[38] Hyndman, R.J. ve Koehler, A.B., (2006), "Another Look at Measures of Forecast Accuracy", International Journal of Forecasting, Vol.22, No.4, pp.679-688.

EK A PROJE YAZILIMININ YAPISI VE ILP MODELİ KODU

Proje Python 3 ile geliştirilmiştir. Veri işlemede polars, SARIMA modellerinde statsmodels, LSTM modelinde PyTorch, ILP modelinde Google OR-Tools ve birim testlerinde pytest kütüphaneleri kullanılmıştır. Tüm sayısal parametreler kodun içine yazılmamış, configs klasöründeki yapılandırma dosyalarından okunmuştur. Rastgelelik içeren tüm adımlarda sabit tohumlar kullanılmıştır. Projenin ilk beş haftadaki temel klasör yapısı aşağıda verilmiştir.

```
MTM4991_Proje/
├── configs/            Yapılandırma: ön işleme, havuzlar, tahmin, ILP, sunucu tipleri
├── data/
│   ├── raw/            Ham Bitbrains ve Alibaba izleri
│   ├── interim/        Ara veri (hizalanmış makine serileri)
│   └── processed/      Havuz düzeyinde 5 dakikalık talep serileri
├── src/
│   ├── ingest.py       Ham izlerin okunması ve hizalanması
│   ├── preprocess.py   Eksik veri kuralları, havuzlar, saatlik tepe talep
│   ├── forecast/       naif.py, sarima.py, lstm.py, ortak.py (ölçütler)
│   ├── ilp/            model.py (ILP, doğrulayıcı, CP-SAT), fiyat_cek.py, solve.py
│   ├── baselines/      tepkisel.py (R0, R1, R2)
│   └── common/         config.py (yapılandırma okuma)
├── tests/              Birim testleri (pytest)
└── outputs/            Çıktılar, tablolar ve şekiller
```

Bölüm 3.5'teki ILP modelini kuran ve çözen fonksiyon aşağıda kısaltılarak verilmiştir (src/ilp/model.py). Yorum satırlarındaki numaralar, kısıtların bu rapordaki eşitlik numaralarıdır. Fonksiyonun ürettiği her plan, kısıtları modelden bağımsız olarak yeniden denetleyen dogrula fonksiyonuyla kontrol edilmektedir.

```
def coz(D_cpu, D_mem, tipler, p, x0=None, y_onceki=None,
        cozucu="SCIP", zaman_siniri_s=60.0, lp=False) -> Plan:
    """Eşitlik 3.23–3.28'i çözer. lp=True: LP gevşetmesi (GLOP)."""
    H, K = len(D_cpu), tipler.K
    x0 = np.zeros(K) if x0 is None else np.asarray(x0, float)
    yd = devralinan(y_onceki, p.L, H)    # önceki günden devralınan açılışlar
    s = pywraplp.Solver.CreateSolver("GLOP" if lp else cozucu)
    s.SetTimeLimit(int(zaman_siniri_s * 1000))
    yeni = s.NumVar if lp else s.IntVar
    x = [[yeni(0, s.infinity(), f"x{k}_{t}") for t in range(H)] for k in range(K)]
    y = [[yeni(0, s.infinity(), f"y{k}_{t}") for t in range(H)] for k in range(K)]
    z = [[yeni(0, s.infinity(), f"z{k}_{t}") for t in range(H)] for k in range(K)]
    for t in range(H):
        s.Add(sum(p.rho_cpu * tipler.C[k] * x[k][t] for k in range(K))
              >= (1 + p.alpha) * D_cpu[t])                          # (3.24)
        s.Add(sum(p.rho_mem * tipler.M[k] * x[k][t] for k in range(K))
              >= (1 + p.alpha) * D_mem[t])                          # (3.25)
        for k in range(K):
            onceki = x0[k] if t == 0 else x[k][t - 1]
            s.Add(x[k][t] - onceki == y[k][t] - z[k][t])            # (3.26)
            L = int(p.L[k])
            s.Add(sum(y[k][tau] for tau in range(max(0, t - L + 1), t + 1))
```

```
              + yd[k, t] <= x[k][t])                                # (3.27)
s.Minimize(sum(tipler.c[k] * x[k][t] + p.s[k] * y[k][t] + p.d[k] * z[k][t]
               for k in range(K) for t in range(H)))                # (3.23)
durum = s.Solve()
if durum != pywraplp.Solver.OPTIMAL:
    raise RuntimeError(f"Çözüm durumu OPTIMAL değil: {durum}")
...                                  # çözümün okunması ve Plan nesnesi
```